Contents

The Narrative Flow of the Project

The project follows a problem-answer-architecture narrative, directly related to how the Scrum technique is used. The problem statement will be divided into sub-questions, each with its own small questions and tasks, resulting in a three-layer structure for the project. The project problem statement is the main layer that will not be explicitly highlighted in each project structure but will serve as the overall theme. The second layer is the major question that needs to be answered, which drives the project forward. The major question will then be divided into four or five sub-questions, and each sub-question will involve a small task that will be saved as a view, or I will call them checkpoints.

In the Scrum technique, I do not build a big, robust structure in one go; I work on small tasks that help me improve and identify the key problems in a specific area of the project.

So, the project works through tasks, and these tasks are mainly saved as views, which can be seen as checkpoints to validate any changes, whether data is added in the future or any new refinements are needed in any of the sub-questions. These views help me work on that sub-question without damaging or significantly influencing the whole structure.

Understanding the Project Problem Statement

The main project problem is determining whether daily financial data can be analyzed and interpreted using ML methods. It is not only about analysis and interpretation; the question is whether these methods can provide meaningful data, relationships, or information that can be applied to future financial data. Can ML models work well and produce meaningful, useful results that can be used further on?

For this major problem, I have used 10 years of daily financial data for 5 major stocks: TCS, ICICI, HDFC, Reliance, and Infosys. In addition to these stocks, I have also included the index daily data. All stock and index daily data include OHLC data, adjusted closing prices, and daily returns. The data spans from November 2015 to October 2025, and in my understanding, 10 years of data is sufficient for the ML model to understand and extract meaningful information that can be applied to future financial data.

So now I will move on to further work on this project using this dataset.

The First Major Question of the Project

After understanding the core problem statement I am going to work on, I now move to finalizing the dataset on which this problem statement will be implemented.

The first major question concerns the fundamentals of ML. ML is about training a machine on historical data, finding patterns in that data, and using that training to estimate the probability of an event that could occur, under the assumption that some relationship exists between past data and future data. ML is not about predicting future data directly; it is about understanding what can happen by analyzing historical data.

One of the core fundamentals of ML, as I define it here, is the feature and the target. A feature is the input given to ML so it can understand the target. The target is the output that ML focuses on, based on the relationship it can have with the features. Deciding the target is extremely important, because if the model is given a vague target that itself has no credibility or significance, then the model will never be useful in producing any meaningful information.

So the first question focuses on the target itself, which is the Nifty 50. I chose it because it is a less volatile index and can also be influenced by these 5 stocks. These 5 stocks make up a major portion of the Nifty 50, so they can influence it more effectively. The problem, then, is whether Nifty 50 returns can be a suitable target for this financial data, and whether they can be treated as the output for this financial data in a way that provides meaningful information that can actually be used.

Sub Question 1 –

After deciding on the target for the financial data, the main question is which ML model should be used as the first lens for this database. The first lens is like a doctor's initial assessment of a person's physical appearance to judge whether the problem they are stating is visible there, or it can be termed the first impression too.

So I ask whether regression can be seen as the first lens for this database. Regression is typically used to determine whether, on average, changes in the independent variables are associated with changes in the dependent variable. In simpler terms, can changes in Y be explained by X in a meaningful way? Can X help us understand the changes that occur in Y?

If regression works in any form with this financial data, it will provide useful information. Even if it does not work, that can also be useful information, because it would indicate

that there is more noise in the market and that the data is more complex, so it will take more effort to extract any meaningful information.

So I conclude that regression can be used on this data for the first checkup. Since I already defined the target, I set the features as the returns of those 5 stocks. I also set the data type to distinguish the train/test split for ML. In the end, I create a view that includes only the elements on which the regression will be performed, because this view itself serves as the checkpoint to understand what happens when I use this data.

Sub Question 2 -

After deciding on the features and target and determining the view on which the regression will be used, the next question is whether the data will yield meaningful results with linear regression. If not, can the failure to produce meaningful results still provide feedback?

As I see it, linear regression is the simplest way to determine whether changes in Y can be deduced from changes in X and whether a very basic relationship between the index and stocks exists. If such a relationship exists, linear regression will show it. If the relationship is more complex, it will not be shown by this method, so I can understand that the relationship cannot be observed in a very direct way.

And the interpretation works like this. What is the actual difference between the train and test results? Specifically, for now, the predicted values are how much more or less than the actual values. The residual matters, and from that I can see whether the results are overfit, where a much more complex relationship exists, as the market cannot be determined so easily. Good fit is where both train and test work well and the test also has a significant R-squared value. If both fail, it can be seen that a more complex relationship is there.

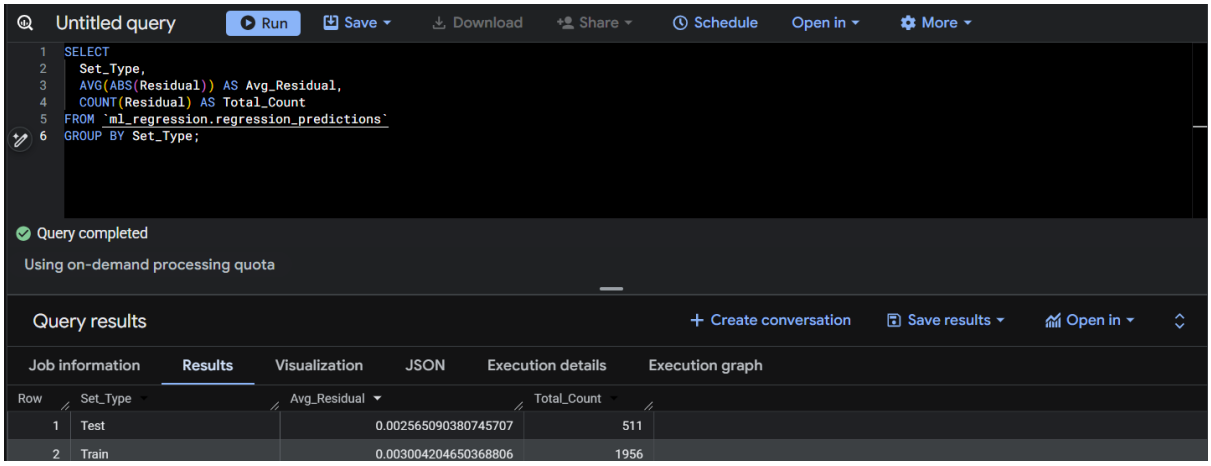
When I make the prediction, there is something more to consider, because the average residual for the training set was higher than the average residual for the test set. The training data was from Nov 2015 to Oct 2022, and the test data was after that up to Oct 2025. The quality of the data also matters, because the training data includes the extreme shock of the COVID period, so the training dataset has experienced much more volatility. By nature, the model focuses on reducing the error on average and capturing meaningful relationships, and that error also includes the COVID period, so it becomes a mix of extreme situations too. By contrast, the test period signifies both a good period and a bad period in 2024, but it was not that bad compared to COVID. So one possible reason is that the training set had to absorb more extreme situations while minimizing error, which can lead to a higher average residual. The main aspect, then, is what to

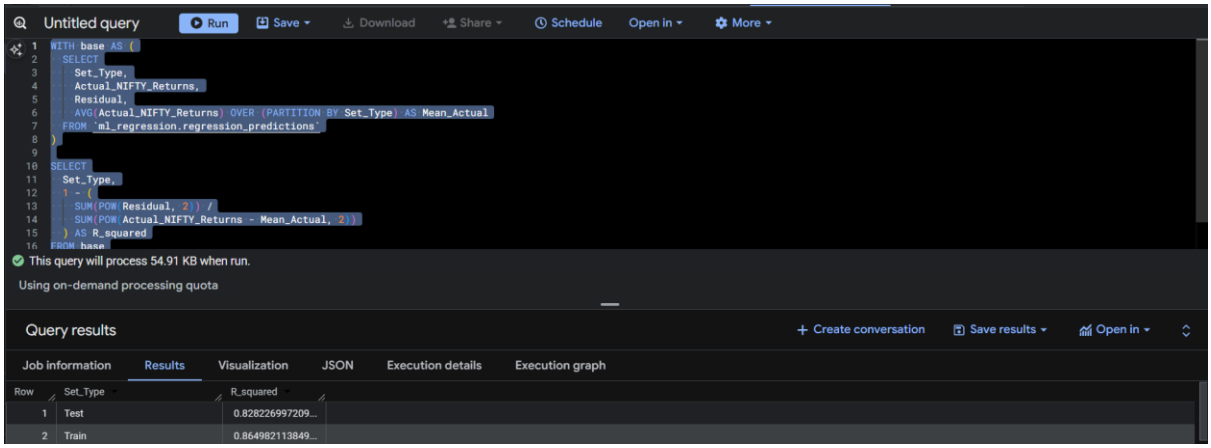
include in the training data. If you want a better result, the training data should include every type of situation in it for sure.

With the R-squared value for the training set at 86% and for the test set at 82%, this also shows that only 82% of the variation in Y is explained by the model on unseen data. At the same time, the lower average residual in the test set can also be understood in the context of lower volatility there.

I should also consider the model data I am working with and the relationship between the target and features. Since Nifty is made up of 50 stocks, and these 5 stocks have a major part in it with much higher weights, this should also be considered when I am modeling the same-day return of Nifty using the same-day returns of these stocks, as these stocks can have a direct influence on the Nifty index.

Overall, the results show a clear and direct relationship between the index and stocks, and linear regression can capture it.





Sub Question 3 -

After performing regression on the specified dataset, the results are very strong, with 86% and 82% R-squared values for training and testing, respectively. Next, I will examine where this model failed and interpret what these failures reveal about market behavior. I will focus on the residuals to gain a clearer understanding of the regression results.

Why should a residual be taken into account? The failure of any model should also be understood by the part it did not explain, not only by what it did explain. In regression, that unexplained part is called the residual. The residual is the gap between the predicted value and the actual value, the part our model was unable to explain. One advantage of a high R-squared is that I can analyze the residuals, see where they are concentrated, and understand their impact.

I will interpret the residuals by how they are distributed. If the residuals are evenly spread across the values, there will not be any issue. However, if the residuals cluster in areas of higher volatility, such as the COVID period in our training data, and the same pattern appears in the test data, it should be examined more thoroughly and treated as a model limitation in understanding extreme volatile periods.

I first focus on the residual statistics, such as the average residual, the average absolute residual, and the maximum absolute residual. The data shows that the average residual for both the train and test sets was close to zero, and the average absolute residual was also close to zero, though slightly higher for the train set, which is normal given the extreme COVID-19 period data in it. The maximum absolute residual was 1.58% for the test set and 2.23% for the train set, reflecting the COVID-19 period volatility, though not by much. The standard deviation was also very close between the two. Overall, this indicates a very strong linear relationship, with no explosions anywhere.

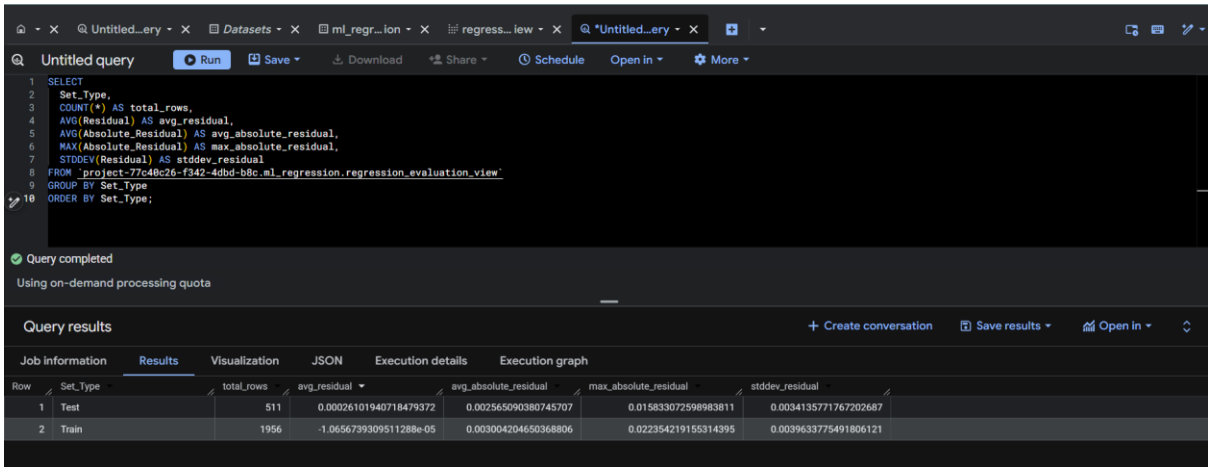
Now I focus on the dates where this extreme volatility occurs and examine it more closely. On the train level, the extreme absolute residuals are not clustered, as I see. Not every extreme volatility event had a very high absolute residual; some were even fitted by the model, which shows that the features I use for the target have a strong relationship, as those stocks also move sharply when Nifty moves. Overall, in the train set, I see evenly spread residual values. The same is true for the test set, with no clustering of residual values or high jumps in the residual during extreme volatility, and even some of those periods were fitted.

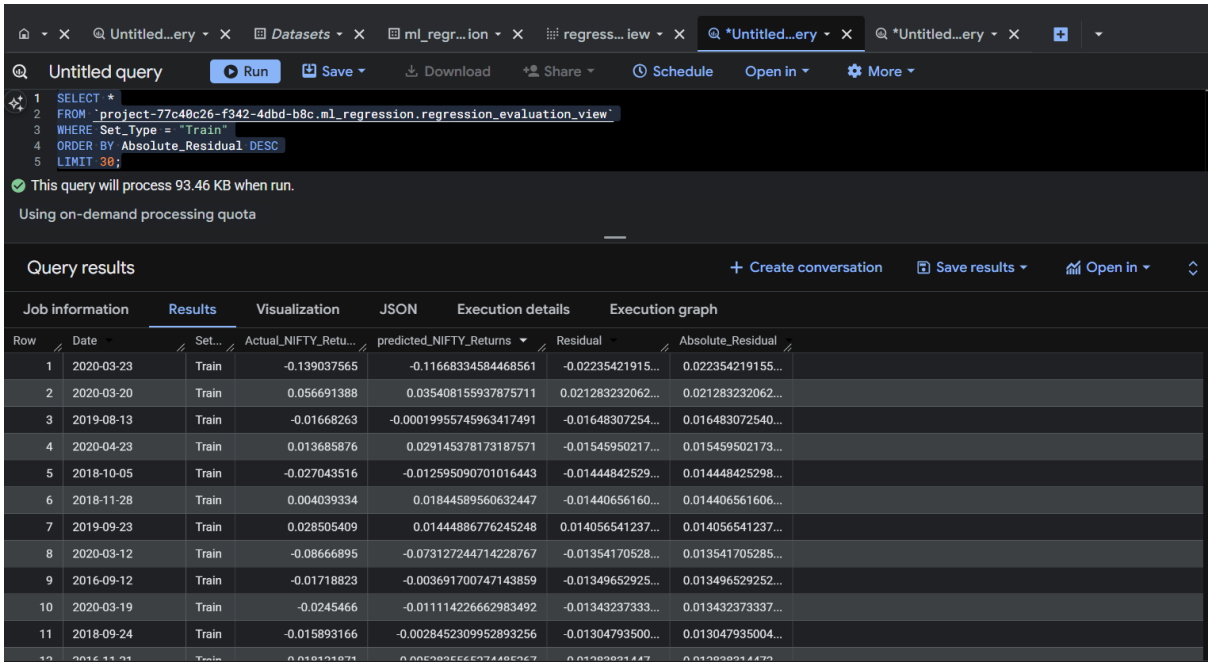
Regarding the final volatility sensitivity check, in simple terms, I am checking whether an increase in Nifty volatility is accompanied by a higher residual. This indicates whether the model struggles during periods of volatility. If the correlation between

volatility and residual is high, it suggests the model fails or underperforms during volatility. If large residuals appear randomly, not only during extreme volatility but also outside it, then it is random error unrelated to volatility, which is healthier for the model.

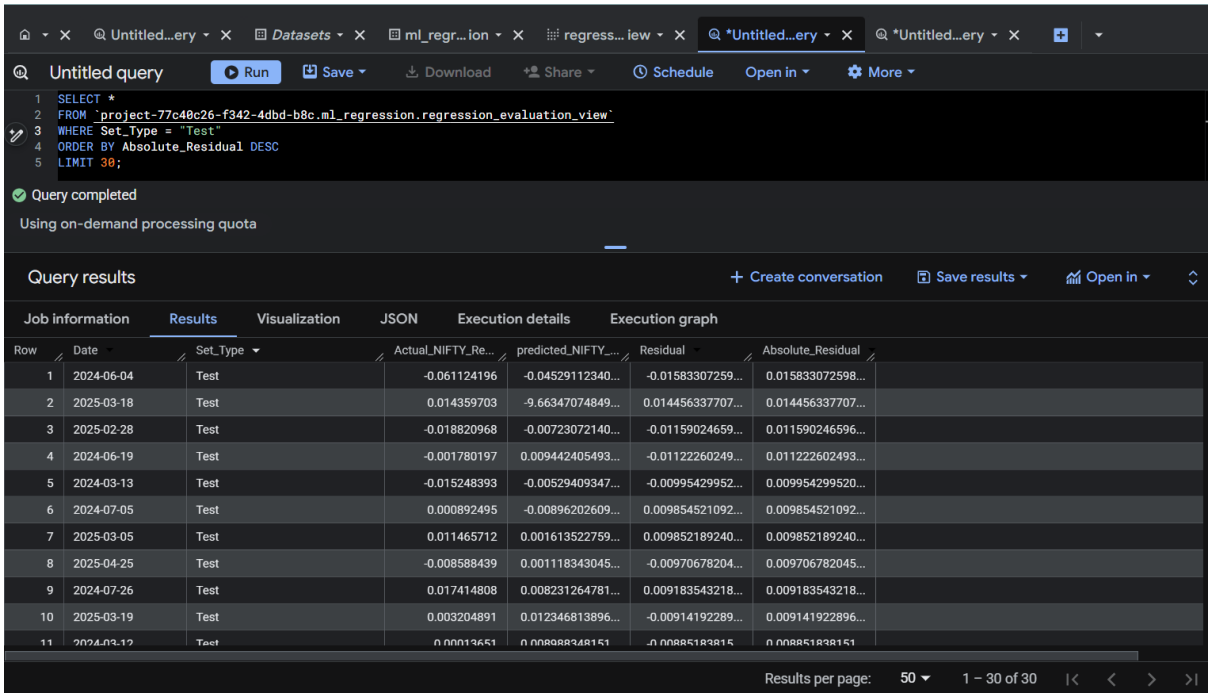
The correlation is 0.35 for the test set and 0.31 for the training set, indicating mild volatility sensitivity that was not obvious from inspecting a few rows. The similarity of these values shows that the relationship is consistent across both datasets. Although the training data includes the COVID period with exceptionally high market volatility, its correlation is only slightly lower than that of the test set. This suggests that the model's volatility sensitivity is present but not driven solely by extreme events such as COVID. Overall, volatility has a mild influence on the model's residuals, but it is not a dominant source of prediction error.

In the end, the model exhibits mild volatility sensitivity but remains structurally stable, as it shows a correlation of around **0.30–0.35**, which is relatively weak. However, the residuals still show a mild positive association with market volatility, indicating that the model's prediction errors tend to increase slightly during more volatile market conditions.

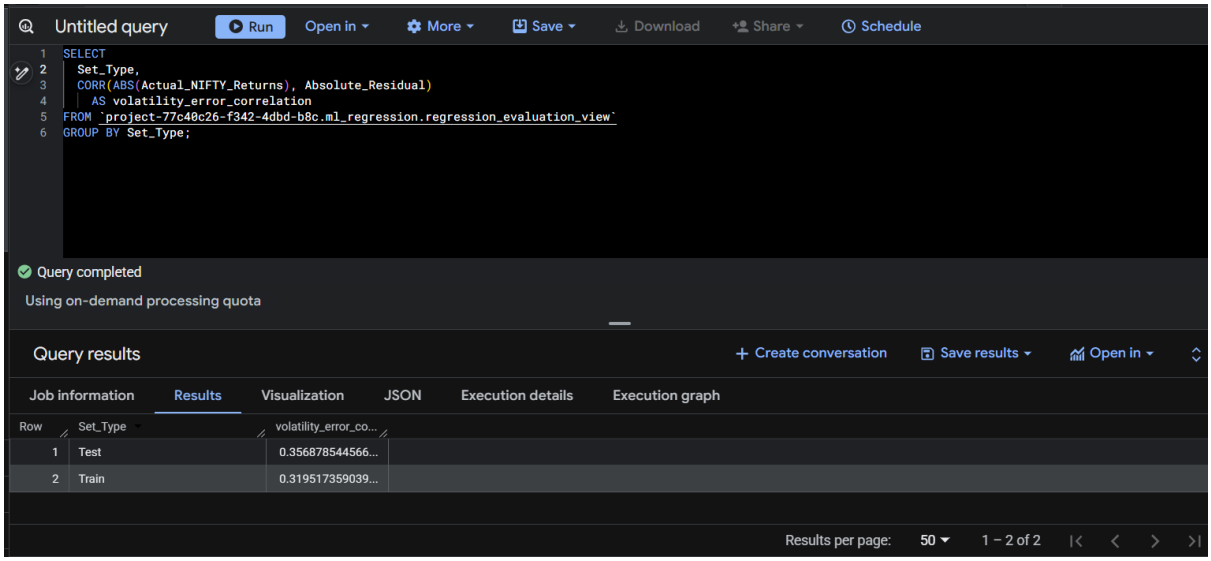




Row	Date	Set...	Actual_NIFTY_Retu...	predicted_NIFTY_Returns	Residual	Absolute_Residual
1	2020-03-23	Train	-0.139037565	-0.11668334584468561	-0.02235421915...	0.022354219155...
2	2020-03-20	Train	0.056691388	0.035408155937875711	0.021283232062...	0.021283232062...
3	2019-08-13	Train	-0.01668263	-0.00019955745963417491	-0.01648307254...	0.016483072540...
4	2020-04-23	Train	0.013685876	0.029145378173187571	-0.01545950217...	0.015459502173...
5	2018-10-05	Train	-0.027043516	-0.012595090701016443	-0.01444842529...	0.014448425298...
6	2018-11-28	Train	0.004039334	0.01844589560632447	-0.01440656160...	0.014406561606...
7	2019-09-23	Train	0.028505409	0.01444886776245248	0.014056541237...	0.014056541237...
8	2020-03-12	Train	-0.08666895	-0.073127244714228767	-0.01354170528...	0.013541705285...
9	2016-09-12	Train	-0.01718823	-0.003691700747143859	-0.01349652925...	0.013496529252...
10	2020-03-19	Train	-0.0245466	-0.011114226662983492	-0.01343237333...	0.013432373337...
11	2018-09-24	Train	-0.015893166	-0.0028452309952893256	-0.01304793500...	0.013047935004...



Row	Date	Set_Type	Actual_NIFTY_Re...	predicted_NIFTY_...	Residual	Absolute_Residual
1	2024-06-04	Test	-0.061124196	-0.04529112340...	-0.01583307259...	0.015833072598...
2	2025-03-18	Test	0.014359703	-9.66347074849...	0.014456337707...	0.014456337707...
3	2025-02-28	Test	-0.018820968	-0.00723072140...	-0.01159024659...	0.011590246596...
4	2024-06-19	Test	-0.001780197	0.009442405493...	-0.01122260249...	0.011222602493...
5	2024-03-13	Test	-0.015248393	-0.00529409347...	-0.00995429952...	0.009954299520...
6	2024-07-05	Test	0.000892495	-0.00896202609...	0.009854521092...	0.009854521092...
7	2025-03-05	Test	0.011465712	0.001613522759...	0.009852189240...	0.009852189240...
8	2025-04-25	Test	-0.008588439	0.001118343045...	-0.00970678204...	0.009706782045...
9	2024-07-26	Test	0.017414808	0.008231264781...	0.009183543218...	0.009183543218...
10	2025-03-19	Test	0.003204891	0.012346813896...	-0.00914192289...	0.009141922896...
11	2024-03-12	Test	0.00013651	0.009988348151	-0.00885183815	0.008851838151



Sub Question 4 -

After identifying where the model falls short, it is still evident that the model is structurally stable. It achieves strong R-squared values, with only a small difference between the training and test results. The average and absolute residuals also remain at normal levels and are consistent across both datasets, indicating stable predictive performance.

The primary limitation is the model's sensitivity to market volatility. The residuals show a mild positive relationship with Nifty volatility, with a correlation of 0.35 for the test set and 0.31 for the training set. Although the training data includes the highly volatile COVID period, the correlation remains close to that of the test set, suggesting that this mild volatility sensitivity is a consistent characteristic of the model rather than being driven solely by extreme market conditions.

This leads to the next question: How useful is the model in practice? Can it be relied upon for real-world applications despite this limitation?

So how can I assess the model's usefulness and judge it? I can examine how the model behaves on a typical day and how much error it produces on that day, which can be measured using MAE (mean absolute error). MAE shows the average magnitude of errors for a typical day, but it can smooth out and underreact to periods of extreme volatility. For extreme volatility, I use RMSE (root mean squared error). RMSE squares the errors before averaging them, so larger errors are penalized more. This makes RMSE more sensitive to extreme days and better suited for capturing volatility, since it reacts more when the residual becomes larger.

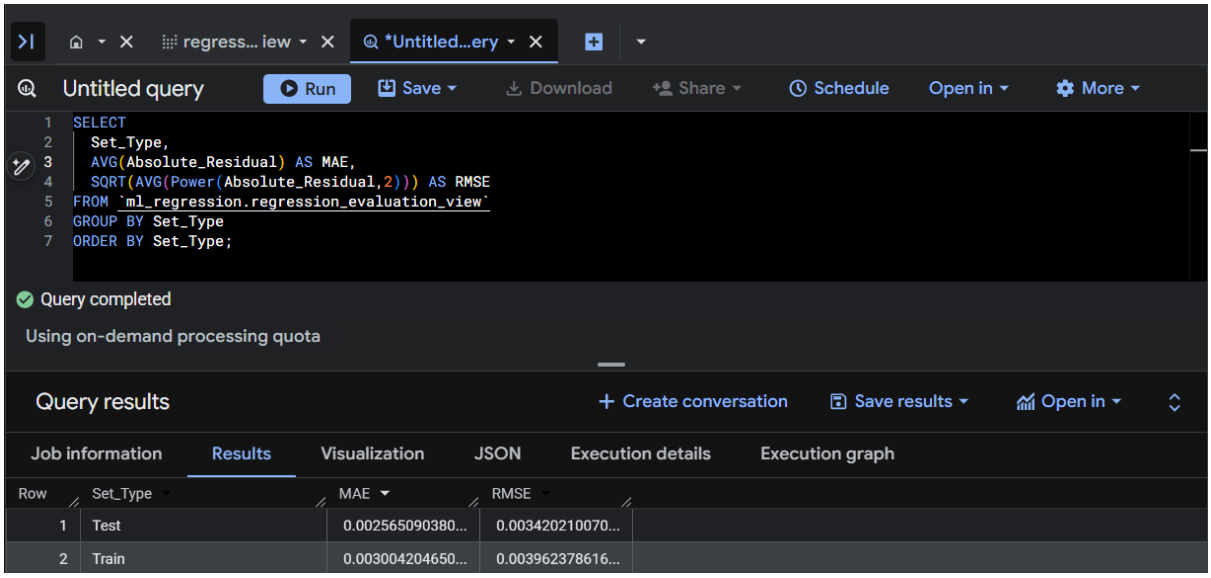
So how should the values of MAE and RMSE be interpreted? The first step is to understand the model's usefulness in market context. Comparing MAE to the data's

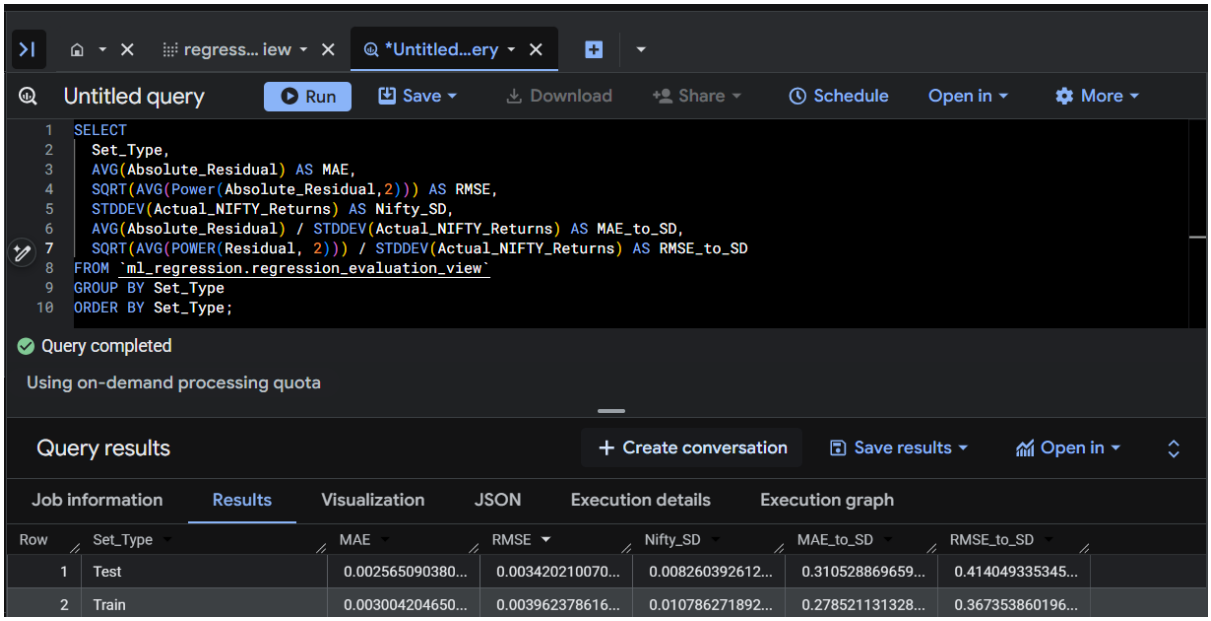
volatility helps. If the error is less than the data's volatility, the model can still be useful, because context matters and the values alone cannot tell the full story. Also, when comparing MAE and RMSE, if RMSE is much larger than MAE, it shows that the model struggles more with volatility, which I had already seen in the earlier correlation result, but now this is visible through the actual values.

It starts with calculating the MAE for the train and test data, as done earlier. The train MAE is higher than the test MAE due to the COVID data in the train period, whereas the test period did not have that same level of extreme volatility, and the values remain close to each other, showing no major deviation. However, the RMSE increases in both the test and train periods, though the increase is not very high compared to the MAE. As with the earlier results, the mild volatility sensitivity is visible here too, because the values do not move too far away from the MAE.

The standard deviation of the train data shows the extreme volatility it had because of the COVID period, whereas the test data was not that volatile. Comparing the SD with the MAE and RMSE, it can be seen that even with the high volatility of Nifty, the model's error remains meaningfully below the overall movement of the dataset. That itself shows the usefulness of the model, because it does not collapse during volatile periods and still performs in a stable way in both the test and train data. Even though the RMSE increases, as the correlation also showed earlier, the actual data now shows that the model still remains within a practical error range when compared with Nifty's daily movement. As this is the final result of the regression, it is saved as a view for future use.

Overall, the results show that the model is useful, as it did not lose its structure and did not collapse during the extreme volatility of the COVID period in the training data. It also reflected a good fit in the training period, with realistic values. The model captures a substantial part of the movement of the actual dataset, which makes it reliable for future use and establishes it as a strong fundamental base for the next part of the project.





Answer to the First Major Question -

The question was whether Nifty_Returns can be used as a target for the historical market data. The answer will be given through the results and analysis that I have done.

First, the R-squared values already indicate a substantial relationship between the features and the target, since the features are the major components that influence the target. The model also shows this relationship with an R-squared value of more than 80%. This shows that not only the target decision, but also the feature selection, was made on a practical basis, by calculating the same-day return of the Nifty with the major stocks that build it and are sure to have a direct influence on it.

As the analysis progresses, the impact of the COVID period becomes evident when comparing the training and test results, particularly through the MAE values. The

training data includes the extreme market volatility experienced during COVID, whereas the test data has not encountered volatility of a similar magnitude. Despite this difference, the model demonstrates a good overall fit, with only minor differences between the training and test performance. The primary limitation is its mild sensitivity to market volatility, as the prediction errors tend to increase during periods of higher volatility. Apart from this limitation, the model produces strong and consistent results, indicating that it remains structurally stable across both datasets.

Regarding the model's usefulness, the comparison of the MAE and RMSE with the data's SD shows that the model error remains meaningfully below the overall movement of the dataset. Even during extreme volatile periods such as COVID, the model did not fail or overfit that period. The stability in the test and train results also shows that the model captured the signal well and was able to carry that structure into the test period.

In simple terms, the model understands the toughest question and its answer from practice, but when the exam question comes slightly easier than what it practiced, the model uses its understanding to answer according to the question, without forcing the extra part it learned during practice.

This type of real understanding forms this model as a very strong baseline structure, and it gives the result of our major question: Nifty_Returns can be used as a target for the historical market data.

The Second Major Question of the Project –

After analyzing the results from the linear regression model, I see that overall the model worked well. The structure was strong, the transition from train to test was stable, and it did not collapse even during the COVID period. From many structural points of view, the model behaved properly.

However, one concern still remains. The model shows mild volatility sensitivity. The correlation between volatility and residual is around 0.30–0.35 in both train and test data. This means that even though the model captures the relationship between the index and its components, the precision of predicting the exact return value is still influenced by volatility.

So the issue is not that regression failed. The issue is that predicting the exact magnitude follows every small fluctuation of the market. When volatility increases, the residual also increases mildly. This suggests that magnitude prediction naturally carries some volatility-driven noise.

This leads me to the next structural question.

If the current target follows every single return value, what happens if I change the structure of the target itself? Instead of predicting the exact return, what if I predict only the direction of the return?

Will that structure be less sensitive to volatility?
Will a directional target reduce the noise that comes from amplitude changes?
And can this reframing give a more stable and decision-relevant signal compared to linear regression?

This becomes the second major question of the project.

Sub Question 1 -

So, after deciding on the second major question, which is what if I change the structure of the target to the directional way, will it reduce the sensitivity to volatility or not, the next question is how I can convert the target, which is continuous in nature, into a directional way, and whether it will be meaningful to convert the target and probably give any better result later on.

The question will be answered through classification. Classification changes the dependent variable into a discrete, categorical variable. For example, house prices can be categorized as high or low, with high defined as greater than or equal to Rs. 1 crore and low as below Rs. 1 crore. This groups the continuous values into categories.

So if I want to categorize the Nifty return, on what parameter should I do it? In practice, returns are usually divided into two broad categories: positive and negative. Most people ask for the direction to understand the whole story or connect the dots. Only when the magnitude is very high do people tend to talk about the real return. For many people, the direction is the ultimate answer for buying or selling a stock rather than the number itself.

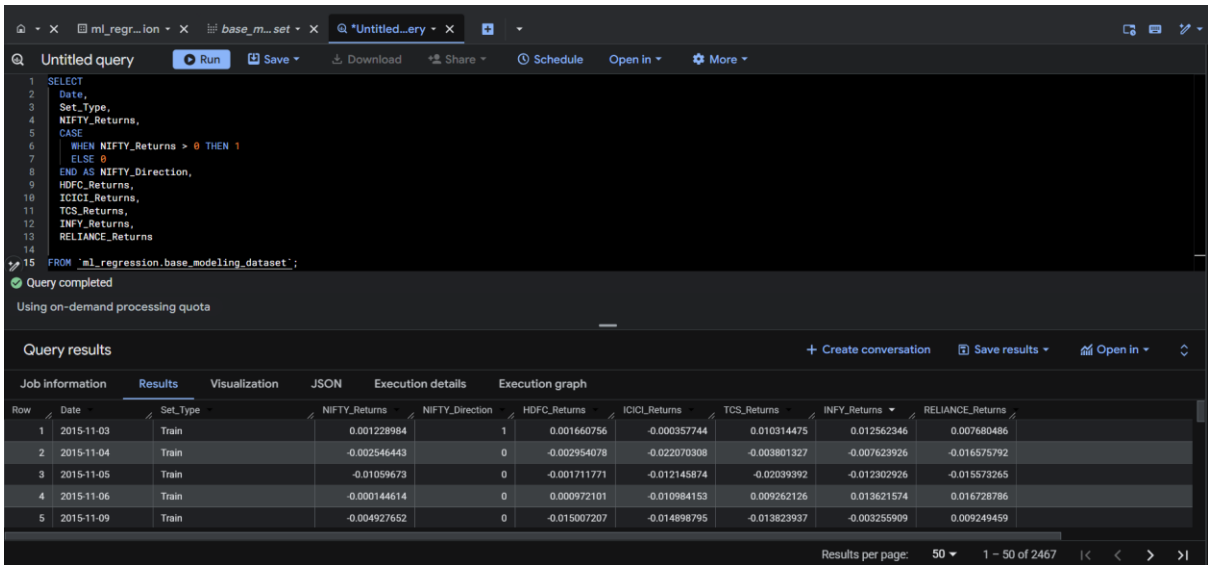
Understanding the categories and putting them into realistic context matters more. In the long term, the market tends to move in a positive direction, so someone can say that most days will be positive. However, on a day-to-day basis, returns tend to move around zero, and there is a 55–45 relationship between the number of positive and negative days, which is important for the classification model. This helps keep the randomness near 50%. If the model moves in only one direction, checking the results will clearly show whether it has done something wrong by not following that balance, or whether it has achieved much better accuracy. But if the data is heavily imbalanced, then accuracy alone will not be meaningful and classification cannot be a better option to proceed. Classification also depends on the time period of the data. If the data is only

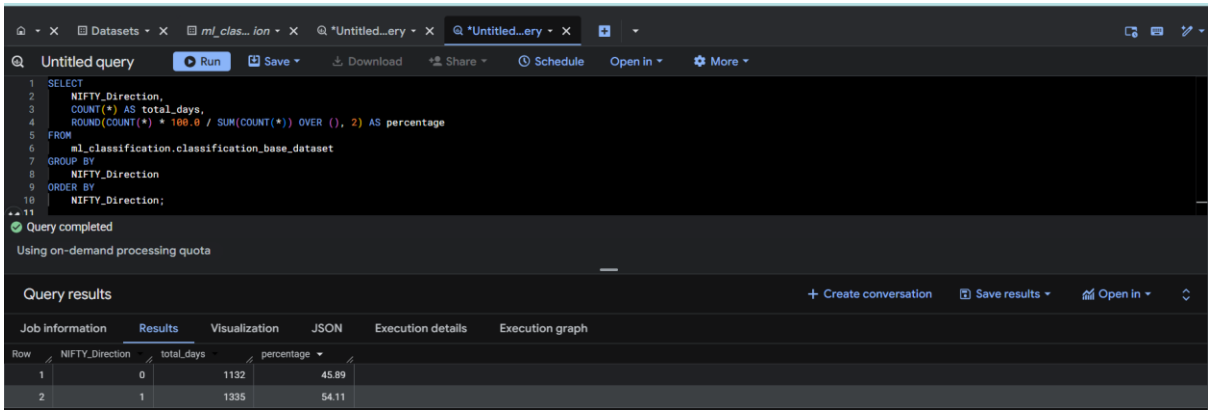
from an extreme period, then the problem is with choosing the data period, not with the method.

So it starts with deriving a new column from nifty_returns, where returns greater than or equal to zero are assigned “1” and others are assigned “0”. This column is named nifty_direction and saved as a view, and now all future analysis will be done on that view until it remains useful.

After this, I try to understand how this category is divided and identify the up days and down days. Up days have “1” in the value, and down days have “0” in the value. The result shows a 55–45 ratio, which is not perfectly balanced, but still balanced enough and not an imbalance. This ratio is good to proceed further with the directional analysis of Nifty returns.

Overall, the ratio shows that the continuous number can be converted meaningfully, and this helps test whether classification can reduce the amplitude-driven volatility sensitivity that was observed in regression.





Sub Question 2 -

After deciding that the directional category is balanced, the next question is whether the data will contain any directional signal from that category. The features remain the same, specifically the returns of the 5 stocks, which are continuous, but the target structure has changed. Can these features still provide any directional signal with respect to the new structure of the target?

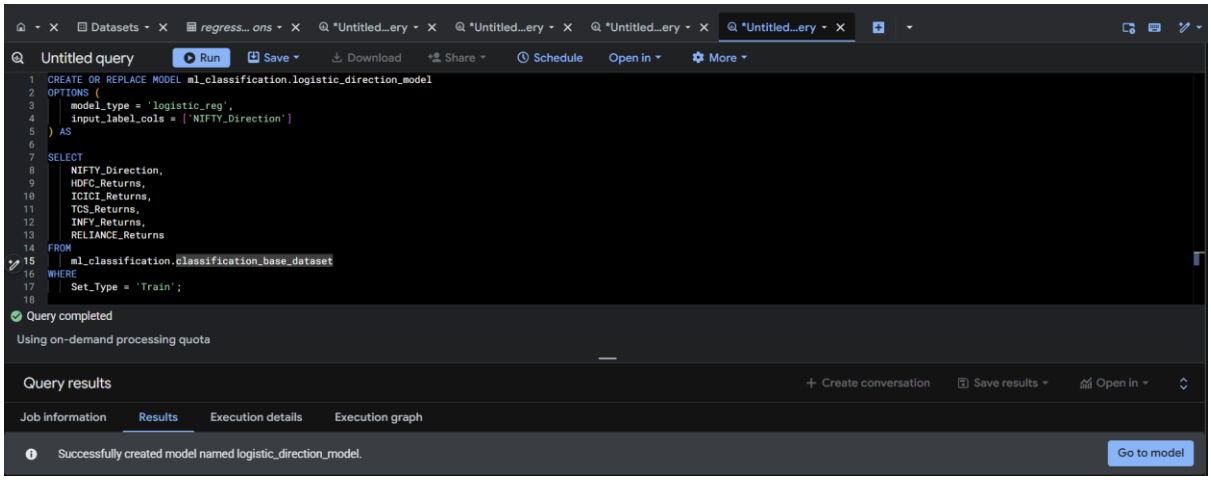
The answer to this question can be obtained using logistic regression. Logistic regression is another type of regression that predicts the probability of a binary outcome. It works like linear regression, forming an equation, but that equation focuses on giving the probability of the outcome rather than a number.

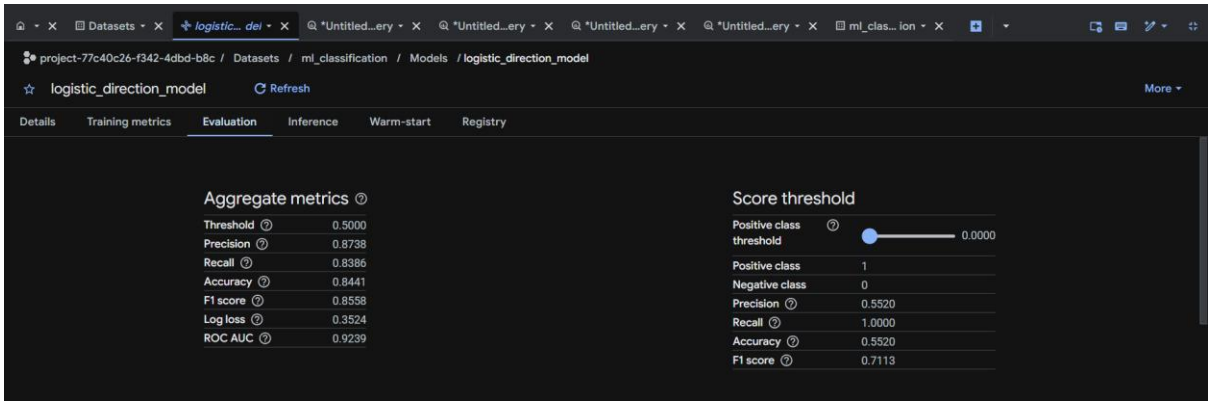
So, how should it be interpreted? The interpretation is done on a row basis and an overall basis. On a row basis, the model provides the probability for each row. The focus is on the spread of these probabilities, which can be visualized with a histogram. If the values are spread across different probabilities, it means the model is making a more conscious decision of its own. But if the values are concentrated near the 50% mark, it shows the model itself is not sure where the direction will move, which points to a weak signal. If there is a spread, the data contains a stronger directional signal.

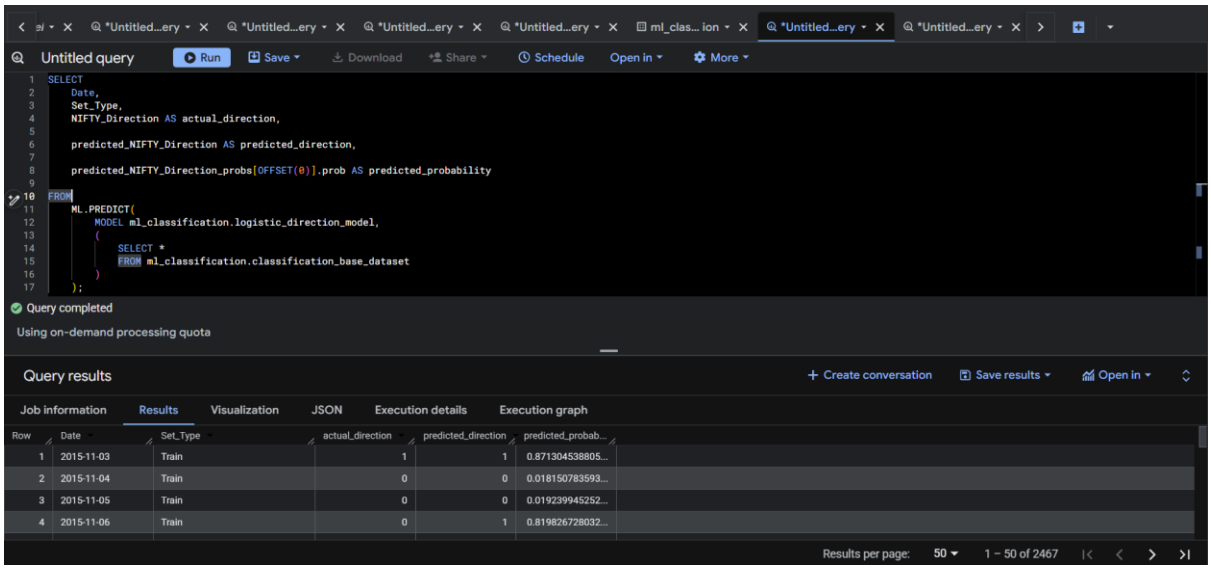
And on the overall basis, our ratio is 55–45. If the overall probability lies in the 55–56 range for up days, then in this project it can be treated as a weak signal, because the model did not learn anything much beyond the natural balance of the data. But if it increases and moves toward 60+, it can show the model learned something better and indicate an overall stronger signal.

Now, the logistic regression model is trained on the training dataset, with nifty_direction as the target and all five stock returns as the features. The resulting model performs well on the training set, with accuracy close to 85%, meaning 85% of the model's predictions are correct. Precision is 87%, meaning that of the model's predictions of "up," how many are actually up. Recall is 83%, meaning that of the total actual "up" values, how many the model correctly predicts. However, the training results should be compared with the test results to assess how well the model generalizes.

Overall, the training results show a very strong directional signal, with all major parameters above 80%, indicating that the data contains a directional signal. The training data confirms this, but it raises another question: what if the test result does not confirm it? This leads to the next question, which is to compare the train and test results row by row.







Sub Question 3 -

After analyzing the results of the training set, the question arises whether the model performs just as well on the test set or whether it has overfit. This will show where the model failed and where it succeeded, and also whether the model is reliable or not.

For the question of where it succeeded and where it failed, the answer can be given by a confusion matrix. A confusion matrix is a table that counts the number of correct and incorrect predictions, and it also shows which predictions the model counts as false. So the table consists of 4 boxes in a 2×2 matrix, and it includes true positive, false positive, true negative, and false negative, telling all the possibilities that can happen when the model predicts the value as positive or negative. This table helps in seeing where the model succeeds and where it fails.

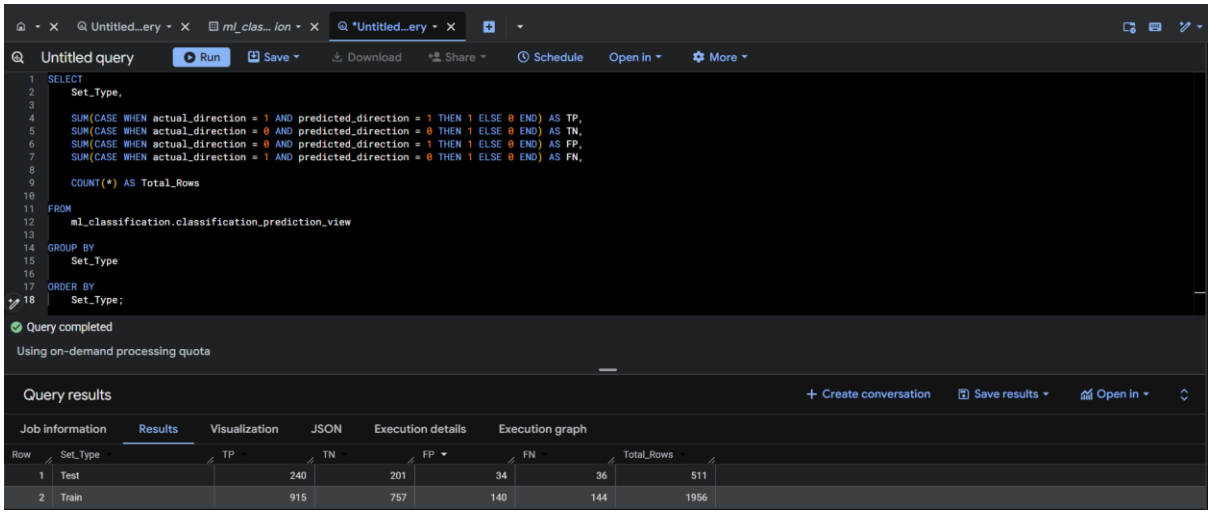
So, how will this confusion matrix be interpreted? The interpretation involves analysis. Did the model show bias toward any specific category, such as producing more false positives and fewer true negatives? If so, it indicates the model is biased toward positive outcomes. In this model, positive is categorized as "Up" and negative as "Down," or as "1" and "0," so the matrix shows this bias clearly. If the matrix shows consistency across the boxes, the model can be seen as balanced. After this, from these boxes, accuracy, precision, and recall will be calculated. Precision tells how many times the model predicted the category and how many times it was correct. This precision will reflect the bias, and recall, which tells how many of the total instances of a category the model predicted, will also indicate the model's bias.

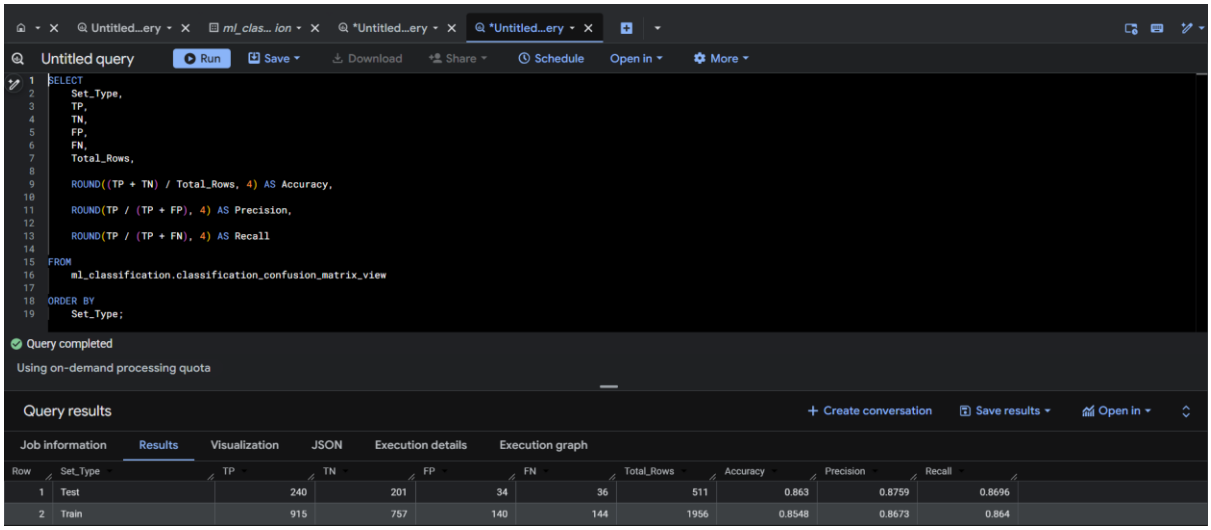
The first step is to obtain the confusion matrix and then use it to compute the accuracy, precision, and recall values. From the values alone, it can be seen that the true positive and true negative are in a 55–45 ratio, which reflects the data structure and indicates that the model is not biased toward any one category. Also, the false positive and false negative are roughly equal in both the training and test data, further showing that the model is not biased toward either category. This also suggests that the model was not overfit, as the test results are roughly symmetrical to the training data. Another possible reason is that the training data included the COVID-period data, which was more extreme, whereas the test data did not have that same extreme period. This pattern was also seen in the linear regression model, so the directional model also behaves well in that aspect.

The second step is to calculate the accuracy, precision, and recall values. After calculating them, there is a difference in the accuracy level. The test accuracy slightly surpasses the training accuracy, and the other values also differ slightly. This is different from the R-squared pattern in the linear model. The main point is the structure. In the linear model, each return was taken as an exact value, but here each return is categorized into two groups, so it is now being seen at a higher level, which gives better stability than the linear model. Since volatility sensitivity is not as much of an issue for a directional model, and there was lower extreme volatility in the test data compared to the training data, the accuracy level slightly surpasses the training data. This is similar to a student who learns and practices with the toughest questions and, in the exam, if

the questions are a little less tough than expected, the accuracy can surpass. This is what the directional model is showing here.

Overall, the answer to the question of where the model succeeds or fails is that no major structural failure is observed at this stage. The model remains balanced across categories, and its test behavior also remains stable. But whether the directional model is more meaningful and useful than the magnitude model will be understood more clearly when I compare the two more directly.





Sub Question 4 -

After analyzing the results and comparing the train and test metrics, it was observed that the test results were equal to the train results and, in some cases, slightly surpassed them. The main questions that arise are whether this directional model is more useful and meaningful than the magnitude model and how both models should be viewed for future use.

The first step is to understand the purpose of both models and how they differ. A linear regression model focuses on magnitude, examining each value and returning a specific value, such as the price of a house or today's return of a stock. It focuses on a specific value, whereas a classification model focuses on categorizing the value. It does not look at a specific value; instead, it focuses on the probability of the category it should fall into, like whether the price of the house is high or low, or whether the return of the stock is positive or negative. It focuses on the probability of the category where the value can go. So both models are different from each other, and this should be considered in the context of which model should be more useful for this dataset.

The interpretation should be grounded in a clear understanding of what the dataset is about and the decisions it supports. The dataset contains the daily historical returns of five stocks and an index, and users of this dataset look ahead to how the market will move tomorrow. Regression predicts the exact return value, whereas classification gives the probability of the positive category. For some, the exact return value may feel more uncertain, whereas the probability is seen as a safer bet. It is important to understand that the features and target are based on the same-day return, meaning the model predicts the same-day return or category of the index using the five stocks' returns from the same day.

In this context, the classification model was more stable, as the test results reduced the volatility sensitivity and produced a more realistic result that was less affected by it. This matters more, because the regression model tends to underperform as volatility increases, whereas the classification model performs better in that aspect. In all other aspects, both models were fairly stable, but as a decision point in extreme volatility conditions, and also in normal cases when asking how the trend is moving, the classification model is better than the regression model.

So how can these two models be compared, given their structural differences? This will be done by 3 principles. The first is stability. In the regression model, overall stability was evident, as the test data R-squared was 82% compared to 86% for the training data. This is generally acceptable in regression models and indicates a good fit. Other metrics like MAE and RMSE did not collapse, so the structure was fairly stable. For the classification model, there was no drop in the test result; in fact, it even slightly surpassed the training result, which is important.

Both models are fundamentally built differently. The training data includes a COVID period, whereas the test data does not have that extreme period. Because the regression model focuses on each return, it tends toward the extreme volatility and tries to fit it into the test data, which does not have that same volatility, so the R-squared drops a little. The classification model categorizes this volatility, so it is seen category-wise. As there was less volatility in the test data compared to the training data, the

directional model score improved, since it focuses on capturing the category rather than the individual return.

The second principle is volatility sensitivity. As the test result of the directional model is equal to or slightly better than the training result, it shows that the model reduces the volatility sensitivity that was driven by magnitude. The third principle is application usage. The regression model is more useful in portfolio valuation, whereas the classification model is more useful for decisions related to buying and selling and for understanding the overall trend of the market, which is more related to the dataset used.

Based on these three principles, it can be concluded that, for this dataset, the classification model appears to be more stable, more meaningful, and more practically useful for future use.

Answer to the Second Major Question -

After analyzing the regression model, the main problem is that it is mildly sensitive to volatility. This sensitivity makes it a less suitable model when the data becomes more volatility-driven in the future. So the question was whether this problem could be reduced by changing the structure of the target, which currently focuses on each return value. Would a new model that follows the direction of the data be able to address this issue?

For the new model, the target was changed to nifty_direction, with “1” assigned to returns greater than or equal to zero and “0” assigned to negative returns. Then, the logistic regression model was used with the same features. The training results were good, with all metrics such as accuracy, precision, and recall remaining stable. But when the test results came up, something important changed. In the regression model, the test result was good at 82% for R-squared compared to 86% in the training data, but here the test result slightly surpassed the training result. All other metrics also showed stability, which directly answers the main problem.

The classification model works on the probability of the category, not on the exact return value, so it reduces the volatility sensitivity that appears in the magnitude-driven model. The training dataset had COVID data, which was much more volatile in nature, whereas the test data did not have that same extreme volatility. So the classification model gives better results with its category-based approach. It is similar to a student who is trained on tough questions, but in the exam the questions come a little less tough. In that case, the overall result can become better than the training result,

because the model is now dealing with categories rather than each individual return value.

So, for this dataset, the classification model reduces the volatility sensitivity that was present in the regression model. Therefore, for this dataset, the classification model tends to be more stable, more meaningful, and more useful than the regression model.

The Third Major Question of the Project -

After running a regression, I obtained a model that was mildly volatility-sensitive, with a correlation of around 0.30–0.35 in both the training and test periods. This volatility sensitivity decreased when the model was replaced with a classification-based, directionally driven model rather than a magnitude-driven one. Because the training data included extreme volatility, such as during COVID, the regression model became more sensitive, whereas the test data did not show that same level of extreme volatility. The regression model did not adjust for this because it focuses on the magnitude of each return, but the classification model handled it better, as it categorized returns rather than focusing on each individual return, which reduced the effect of volatility sensitivity. This was also supported by the fact that the test results slightly surpassed the training results.

Another curiosity arises after seeing this result. If a two-category binary classification model can give stable and strong results under a linear probability structure, what if the directional relationship between the features and the target is not uniform across all market conditions? Would a model that allows conditional or segmented behavior capture the structure differently, or would it make the results noisier and less stable than the classification model itself?

For this new curiosity, the model now tends to move toward a nonlinear approach, because a linear model works with one consistent rule for the entire dataset and does not adjust its structure across different market regimes.

Sub Question 1 -

So, why a non-linear model, and what type of non-linear model should I go with? Looking at the dataset, it has multiple stocks as features, and the stocks themselves belong to different sectors. Two stocks are from the banking sector, another two are from the IT sector, and the last stock, Reliance, is a mix of many sectors. Also, when I look at the dataset period, the train data includes an extreme period, while the test data

did not have that same level of extreme volatility. Because of these sector differences and the change in volatility, a non-linear model becomes worth checking more closely.

Taking the decision tree into account for this makes sense. Since there are two sectors with two stocks each, new conditions based on those stocks' returns may help explain the target better, and sometimes ignoring one part and focusing on another part may also help. A decision tree can make such rules and conditions by checking how the target moves under different feature conditions. This is why it becomes useful here. It can help capture whether, under some conditions, one stock matters more, or whether a combination of two or three stocks matters more than using all five in the same straight-line way.

So how do decision tree splits work in this dataset? The splits create subsets that better explain the target's direction or value, and the tree focuses on the subset that provides greater clarity. It keeps splitting that subset into smaller subsets until the target value becomes as clear as possible. Using this dataset as an example, I have five stocks. The decision tree may start by splitting on one stock, like HDFC, and then examine the target value. After that, it checks the other stocks to see which movements align better with the target's direction, and it selects the next stock to split further. In this way, it keeps adding conditions and checking which combination increases the clarity of the target value.

The decision tree has its own advantage and disadvantage, and both depend a lot on how the user controls it. As the name suggests, a decision tree is a tree. If I visualize a tree, no one can fully trace every branch and endpoint once it becomes too large. If a tree is small, the branches and leaves can still be followed, but as it grows, tracing everything becomes impossible. The same is true for a decision tree. It also has branches, called nodes, and the endpoints of those nodes are leaves. This is where both the advantage and disadvantage come in. If a decision tree is allowed to grow too freely, it will try to connect every single data point, including noise, and then the model can become overfitted. But if the tree is controlled properly and analyzed well, it can provide better results by capturing the signals more clearly. So in the end, it comes down to how well the decision tree is controlled and how well its depth is adjusted to understand the dataset signals.

First, I want to train the model in a freer-growth style to check whether the signal is mostly uniform, as the linear models were suggesting, or whether there is some nonlinearity, such as one stock or one sector predicting the target better than others, or whether combining two or three stocks inside a node helps predict the target better than using all five in one straight structure. I also want to see whether this freer-growth style starts overfitting or not.

Because the whole project is on the BigQuery ML platform, there are some limitations. The first limitation I found is that BigQuery ML does not directly support a standalone decision tree in the way I wanted to test it, so I moved to boosted trees. Boosted trees are an ensemble. It is like many judges analyzing the result, where each later judge focuses more on the mistakes or gaps left by the earlier one. In this way, the next tree works more on the areas where the earlier tree struggled, and the final result becomes a combined model rather than a single tree. Since I still wanted to study tree-like conditional behavior, I used the depth settings in a more controlled way so that the model stays closer to that type of structure, even though technically it is still an ensemble model. This also helps control overfitting.

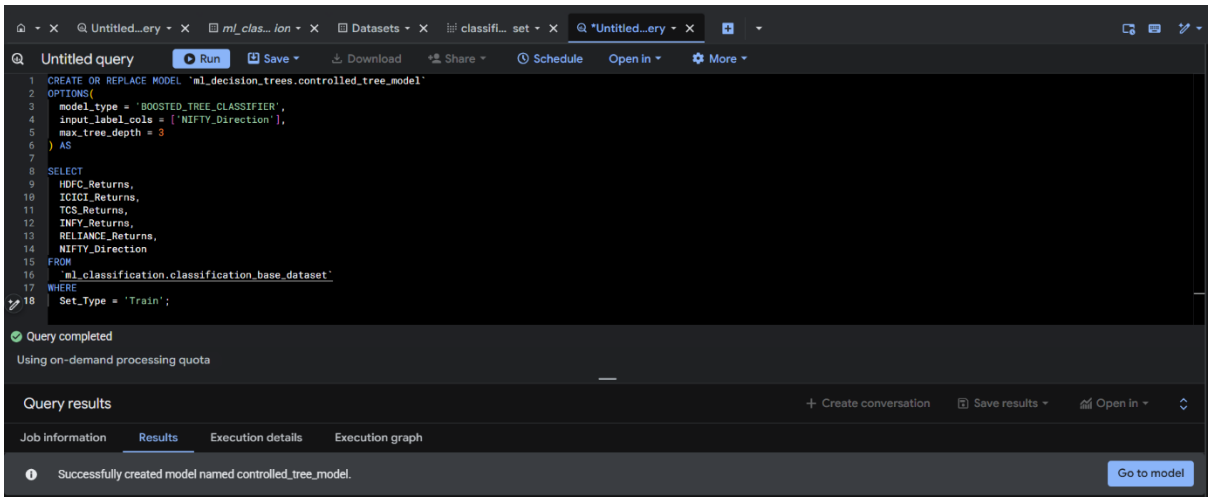
The model results show something important. First, the model tries to build 20 trees but stops at 11 because no further improvement is observed. The final accuracy of 84.5% is slightly lower than the training classification accuracy, which means the model did not improve at the training level itself. This suggests that the directional signal may still be mostly linear. The consistency in precision and recall shows that the model remains stable. The test results will confirm whether this model is actually a good fit or not, but for now the directional signal looks more linear than strongly nonlinear.

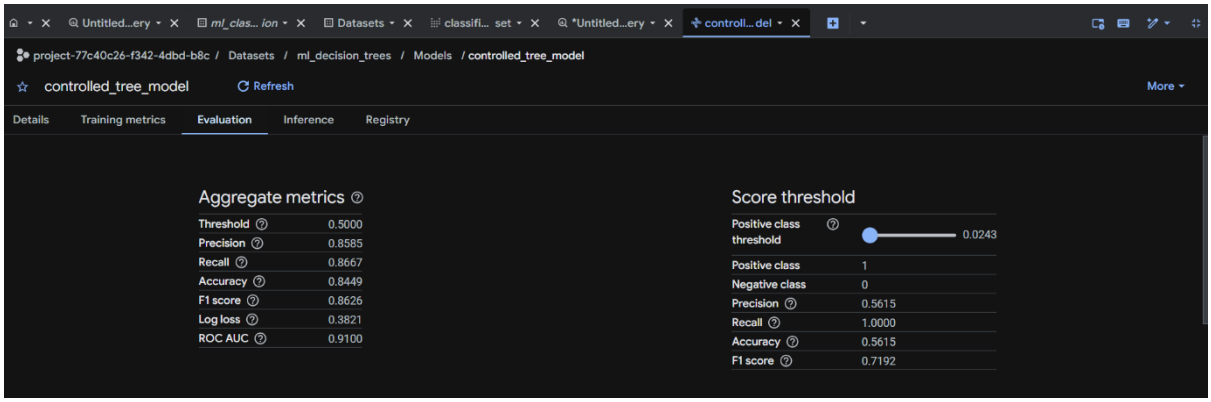
Before moving to the test predictions, I also look at the features and where the splits are happening. Are there any sectors or stocks that create major splits? Is there any conditional behavior where combinations of two or three features affect the target more strongly or not?

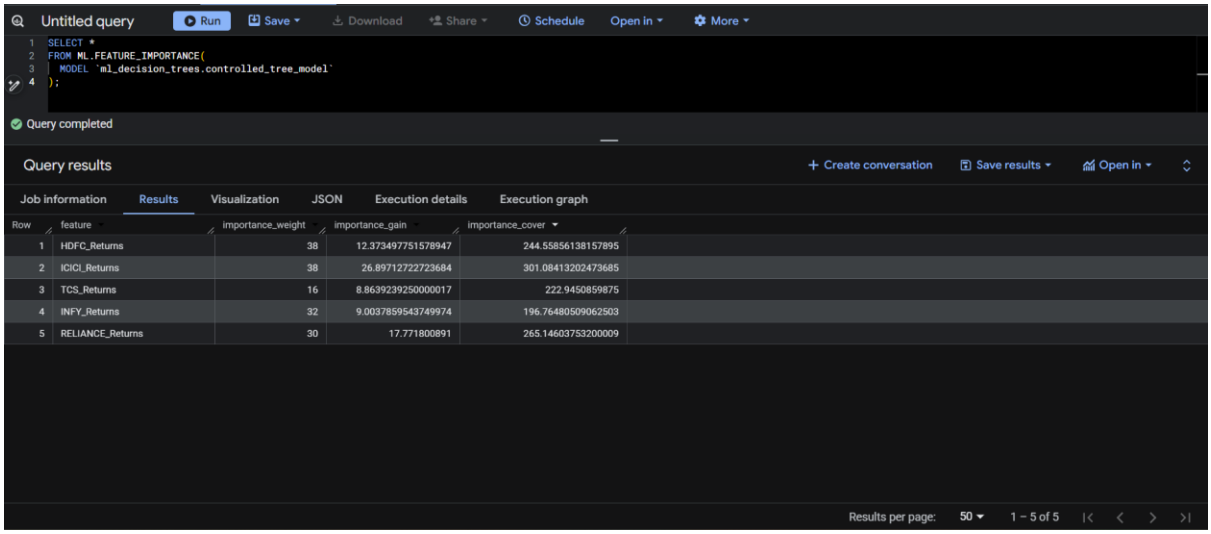
The results show some contrasts that help explain why trees are worth checking. Importance gain is the most important metric from these results. Importance gain tells us whether the splits are actually reducing Gini impurity. Reducing Gini impurity means the split is moving toward a clearer signal in one direction rather than staying in a mix of noise and signal. In simple terms, it reduces confusion and clears some noise. Importance weight tells us how many times a feature is used during splitting. From the results, even features that were used more often did not always provide more efficiency, because those splits did not always reduce Gini impurity by much.

For example, between HDFC and ICICI, both were used most often in the splits, but ICICI splits were much more effective in reducing Gini impurity, so those splits were more useful than HDFC. The same pattern appears between Reliance and Infosys. Both were used similarly in splits, but Reliance gave a much higher importance gain than Infosys. So from this, it can be seen that ICICI and Reliance splits were more effective than some other features. It is like in cricket when two players face a similar number of balls, but one creates much more impact from them.

Overall, this result shows that the split usage was distributed across features, but some features were more effective than others. There is no single-feature dominance, and the results also show that combining different conditions can increase the impact. However, the accuracy remains closely aligned with the classification result, and that suggests that some conditional effect is present, but it is not strong enough to improve the model clearly at this stage. A comparison of the test and train results will give greater clarity and will show whether the model actually learned something more or not. It will also help answer whether the directional signal is mostly linear or meaningfully nonlinear, especially since the test dataset is less volatile in nature.







Sub Question 2 -

After understanding why a tree model should be used, it becomes important to understand what exactly I should look for inside the tree model. From the earlier result, it was evident that some features are more impactful than others, but no single feature dominates the whole structure. This means the model is not being driven by only one feature, but by a more distributed pattern. So the main question now is what I should try to understand from a tree model. A tree model works on splitting, or segmentation. So how does the tree perform that splitting, and what are the major things it considers before making each split? What should be understood about this dataset through this model?

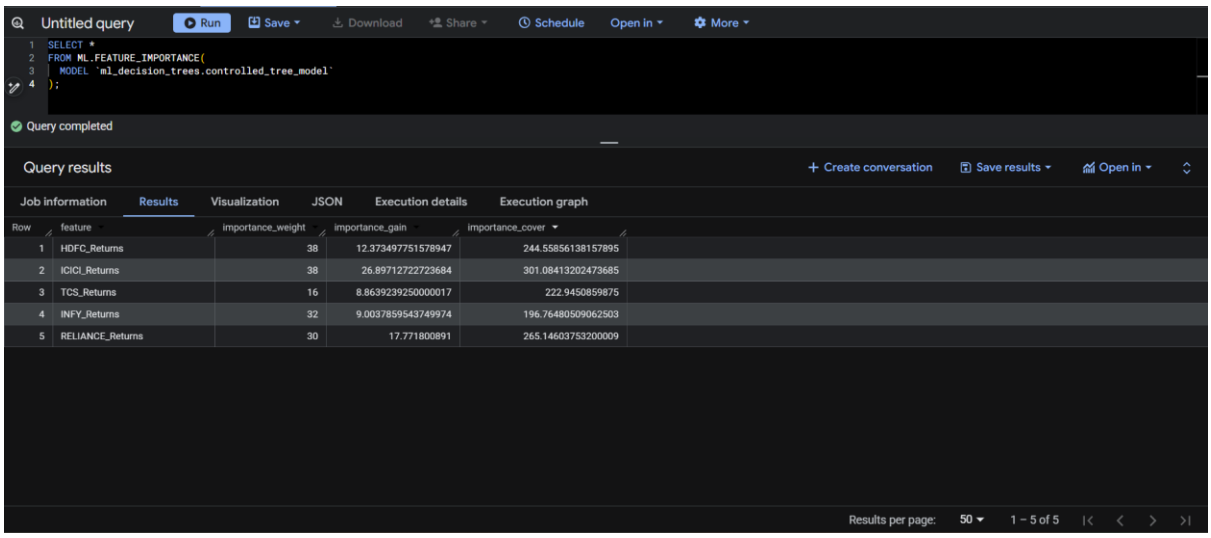
The best way to understand how the tree works and what it considers before making a split is through Gini impurity. I have discussed it before, but this is the right place to focus on it properly. Gini impurity basically measures how much confusion is present in the data before the split, or how mixed the outcomes are. If the probability is 50–50, that is the most confusing case, because no clear answer is visible there. But if the outcome becomes more one-sided, the confusion reduces. For example, instead of 50–50, if the result is 80–20, then the direction becomes much clearer. This is what the tree focuses on before making a split. It tries to find the right condition or the right feature that reduces that confusion. In tree language, this is called reducing Gini impurity. So the lower the impurity value, the better. The tree prefers those splits that reduce confusion more effectively.

The result from the tree model, especially feature importance as weightage and importance gain, becomes the basis of this analysis. The reduction in Gini impurity is one of the most important things the tree focuses on, and in the result this is reflected through the importance gain. It can be seen that ICICI has the highest gain value, more

than double that of HDFC, and this is reflected in the split behavior. Importance weight shows how many times a feature is used in the splits. From this, it is visible that HDFC and ICICI were used the most, but ICICI was much more effective in reducing Gini impurity than HDFC.

After ICICI, RELIANCE also appears to be highly effective. Even if it is used less in the formation of splits, its effectiveness is high compared with some other features. On the other hand, INFY and HDFC may appear more often, but they do not provide the same effectiveness, because their splits are not reducing the impurity as strongly as ICICI and RELIANCE. So the result suggests that ICICI is one of the strongest candidates for major splits in this dataset, and RELIANCE also adds strong conditional value when the model is trying to reduce confusion further.

Overall, this question helps explain how the tree model works and how it identifies which features are more effective than others. For this dataset, in the training data with a maximum depth of 3, depth means how many splits can happen from the root to the leaf. A leaf is the final result where no further splits are made, so a maximum depth of 3 means there can be at most 3 splits from start to end along one path. Given these factors, and also the BigQuery ML limitation, the result suggests that ICICI is the most effective feature in helping the tree reduce confusion, with RELIANCE also showing strong usefulness. Now the next major question becomes how the model behaves on the test data. Did it overfit, or did it actually give a better result? That should be the next question.



Sub Question 3 -

So now the question arises about the model's applicability. Can it be sustained on test data or not? Will it produce better results like the classification model did, or will it overfit? Or will it be a good fit, but not as good as the classification result was?

So what is overfitting in trees? Trees work by forming nodes at each split, so in a small tree you can see where a branch starts and where it ends. But in a big, full-grown tree, it becomes impossible to trace the path because the branches are deeper and denser. These dense branches can be created for very few data points, and that is what overfitting in a tree means. It keeps making splits until it starts trying to match each value too closely.

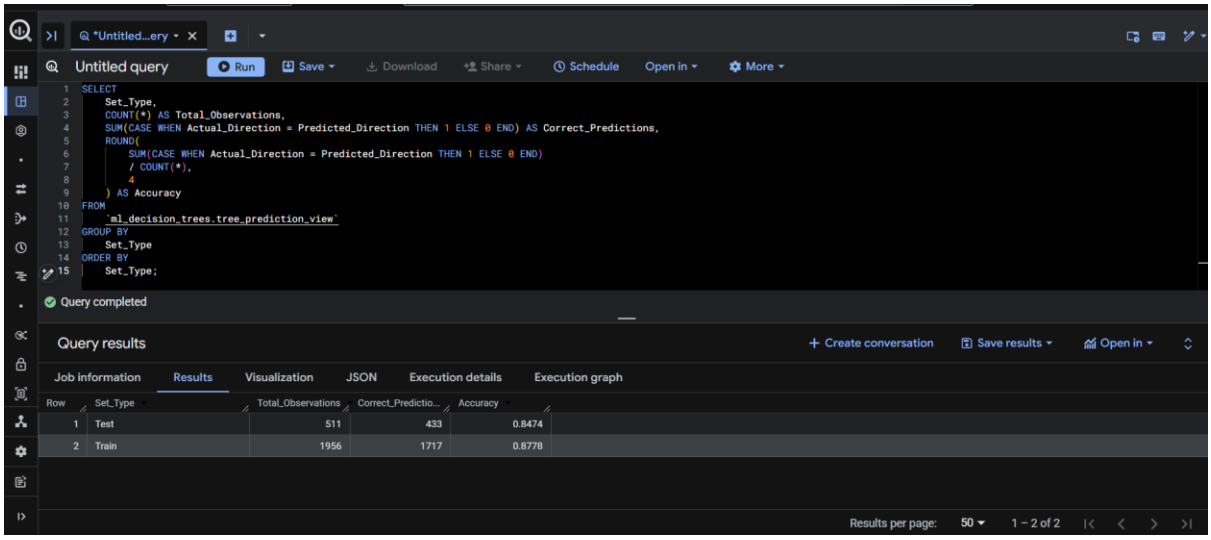
That is what an unrestricted tree can do, but because of the BigQuery ML limitation of not supporting a single tree directly in the way I wanted, that unrestricted single-tree version was not available. And since I am focusing here on the fundamentals and basis of nonlinear models, the model I am using is a restricted ensemble tree with a max depth of 3. So it is not that dense. It is a shallow tree, which means it can generalize more, as it tries to capture broader themes rather than deep single-point themes, and because of that the chance of overfitting becomes lower.

The prediction results show approximately 88% for training accuracy and 85% for test accuracy. With these close values and only a small gap, the model is stable, or I can say it is a good fit. Such small gaps are normal in a model, and this shows that the model did not collapse at all on test data.

But this also raises the next part of the comparison with the classification model. The tree model did not produce the same kind of test strength that was seen in the classification model, where the test result slightly surpassed the training result. So overall, the directional signal still seems to follow one broader rule, or in simple terms, it still appears mostly linear in nature.

Under the current structure of this tree model, one major reason can also be the depth value of 3. That depth setting pushes the model to focus more on the broader theme, and that broader theme may already be captured well by the linear model itself. So from this result, it can be understood that some conditional effect may be present, but it is not strong enough at this stage to clearly outperform the classification model.

This result opens more questions for me, and those will be analyzed further. But for now, the main conclusion from this question is that the tree model gives a good fit, does not overfit, and remains stable on the test data, though it does not improve enough to show a strong nonlinear advantage over the classification model.



Sub Question 4 -

After evaluating the results on the training and test datasets, the analysis shows that the model is stable and a good fit, as it did not collapse on the test data. However, the main question now is about its usefulness and applicability. Did the model perform better than the existing classification model? Classification performed very well compared with regression. So does this tree model do anything extra or not? Is there any real usefulness in this model, or has the classification model already explained most of the directional signal? This is the main question: whether this tree model adds anything useful to the understanding of the dataset, and whether the directional signal is mostly linear or not.

To answer this question, the first key point is understanding the structural difference between a linear model and a nonlinear model. Even though the linear model I am comparing here is a classification model, its structure is still linear in nature. It applies a single rule across the dataset, or one equation, and that rule defines a consistent effect on the target. That effect remains the same no matter the situation. That is how a linear model works. A classification model also follows this structure, but instead of targeting each data point as an exact value, it focuses on the binary decision of up and down, or 1 and 0. In simple terms, it makes categories for the dataset. So it works at the level of a theme, which reduces the individual effect because it focuses on the directional structure of the dataset.

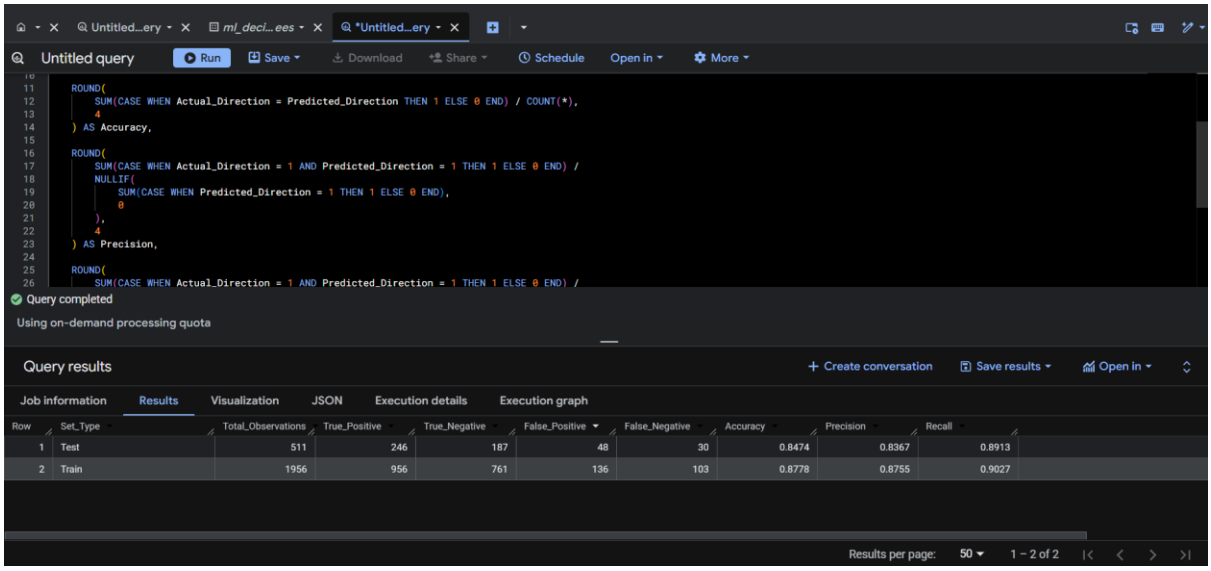
Now, what about a nonlinear model? A nonlinear model makes conditional rules depending on the situation. It can focus on how different segments of a feature affect the target and then split those segments, making a conditional rule for each segment. So it is like one common rule versus many conditional rules. In nonlinear modeling here, I am using a tree model, which makes splits to form those conditional rules.

For this, I followed the same approach as in the classification metrics, computing all major values, not only accuracy but also precision and recall, along with a confusion matrix to understand the overall model results. As shown in the results, all metrics are stable and well fit, with no collapse on the test data. The values of false positives and false negatives are low. Percentage-wise, they are around 15% of the total positives and negatives in both the train and test data, which shows that the model did not become biased in any one direction. This is also evident from the high number of correct predictions it made on both the positive and negative sides, so the model did not show bias toward any single direction, which makes it more stable. Recall is one of the strongest values here, as it is around 90% for both the test and train data, which shows that the model is strong at capturing the actual category.

Now comes the real usefulness question. With all the extra complexity this model has, is it worth using or not? The main answer comes from the accuracy result and the other metrics. In the classification model, every test metric slightly surpassed the training result. But in this nonlinear model, every test metric is lower than the training metric. At the same time, the overall results remain quite close to the logistic regression model. This shows that the directional signal is still mostly linear in nature, because this model did not extend the classification result in any meaningful way. And if the same thing can be explained and analyzed by a simpler model, then adding more complexity without added usefulness does not help much.

So overall, the answer is not in favor of the tree model in its current form. It remains stable, but it does not add anything beyond what I already have in the classification model. However, it does raise a new question: what if increasing the depth shows a stronger nonlinear signal, or what if the linearity itself is more complex in nature and can only be understood through a deeper ensemble tree structure?

Overall, the tree model with a maximum depth of 3 does not seem to be more useful than the classification model, as it did not add anything beyond what I already had from classification. But it definitely raises further curiosity about whether the signal is more complex in nature and can only be identified by adding more complexity to the model, such as increasing the depth value.



Answer to the third major question -

The third major question was whether the directional relationship between the features and the target remains uniform across all market conditions, or whether some conditional or segmented structure exists that can be captured better by a nonlinear model.

To examine this, I moved from the linear classification model toward a tree-based structure. However, one important practical limitation was that BigQuery ML did not support a standalone single decision tree in the way I wanted to test it. Because of that, the model I used was technically an ensemble tree by nature. Still, since my purpose at this stage was to understand the fundamentals of decision-tree-type behavior first, I kept the maximum depth at 3 and treated it as a tree model in the project structure, SQL query, and views. The idea was not to claim that it was a pure single decision tree, but to use a shallow ensemble structure in a way that behaves closer to a controlled tree-style model, so I could first understand whether any conditional nonlinear signal exists in the dataset or not.

After applying this model, the results showed that the model was stable and did not collapse on the test data. The train and test results remained close, which means the model did not overfit under the current structure. This itself is useful, because it shows that even when conditional splitting is introduced, the model remains controlled and interpretable at this depth.

However, the more important question was whether this nonlinear structure added anything meaningfully beyond the classification model. Here, the answer is more limited. The tree model remained stable, but it did not improve enough over the classification model to show a strong nonlinear advantage. The classification model

had already shown very strong and stable directional results, and this tree-based structure, even after introducing conditional splits, remained quite close to it. This suggests that the directional signal in the dataset is still mostly linear in nature, at least under the current depth and model setting.

At the same time, the tree analysis did show that some features were more effective than others in reducing confusion, especially ICICI and Reliance, which means that some conditional effect is present in the data. But that effect was not strong enough to clearly extend the classification result. So the answer is not that nonlinear behavior is absent, but that it is not yet strong enough, under this current setup, to make the tree model more useful than the simpler classification model.

So overall, the third major question shows that a shallow tree-style nonlinear approach can remain stable and can reveal some conditional structure in the data, but under the current BigQuery ML limitation and depth setting, it does not add enough extra usefulness beyond the classification model. Therefore, the directional signal still appears mostly linear at this stage, even though some conditional nonlinear traces are visible.

The Fourth Major Question of the Project -

The third major question was about introducing a non-linear model into the dataset to understand whether any conditional rules exist, because in the real world the stock market does not always follow one simple rule. Since the dataset includes the historical data of the Nifty index and the five major stocks that strongly influence it, the purpose of that question was to check whether there is any non-uniformity in the directional signal of the dataset.

The initial plan was to use a single decision tree and bring decision-tree logic into the dataset. However, because of the limitation in BigQuery ML, which does not support a standalone single decision tree in the way I wanted to test it, I had to use an ensemble tree structure instead. Since even trying to make it behave like a single tree in a direct way was not properly supported, I limited the depth to a maximum of 3. This was my first step toward understanding non-linearity through a shallow tree-style structure.

The results were stable. The model did not collapse on the test data, and the major metrics remained well fit. However, when I compared it with the logistic regression model, the usefulness of that shallow tree structure remained limited. It did not add enough new understanding, and it did not clearly show why this model should be preferred over the simpler classification model.

This leads to the fourth major question. If I now increase the depth and bring in the ensemble tree more directly for what it really is, will it provide more useful information? Can it capture stronger non-linearity in the dataset, or will it only start overfitting? More complexity tends to move the model away from broad structure and closer to individual data points. In simple terms, adding depth turns a small tree into a much larger one, with many more branches and leaves. So the fourth major question is whether the boosted ensemble tree can reveal any hidden non-linear structure in the dataset, or whether the directional signal is still mostly linear and extra complexity only adds overfitting.

Sub Question 1 -

Another question that arises when moving forward with the ensemble model is: why combine multiple models, and what benefit does that offer over a single model? It is also important to understand the fundamentals of combining multiple models before applying that structure to the dataset.

So, what is ensemble learning, and how does that model work? Ensemble means combining many small models to produce a more stable or reliable output. A single model may focus on one or two rules, but it may not capture every pattern in the dataset. So using many models together becomes a better way to reduce the limitation of a single model. By combining multiple models and then using their results to make predictions, either through voting in classification or averaging in regression, the final output becomes more stable. Overall, ensemble models help make predictions more stable and reliable.

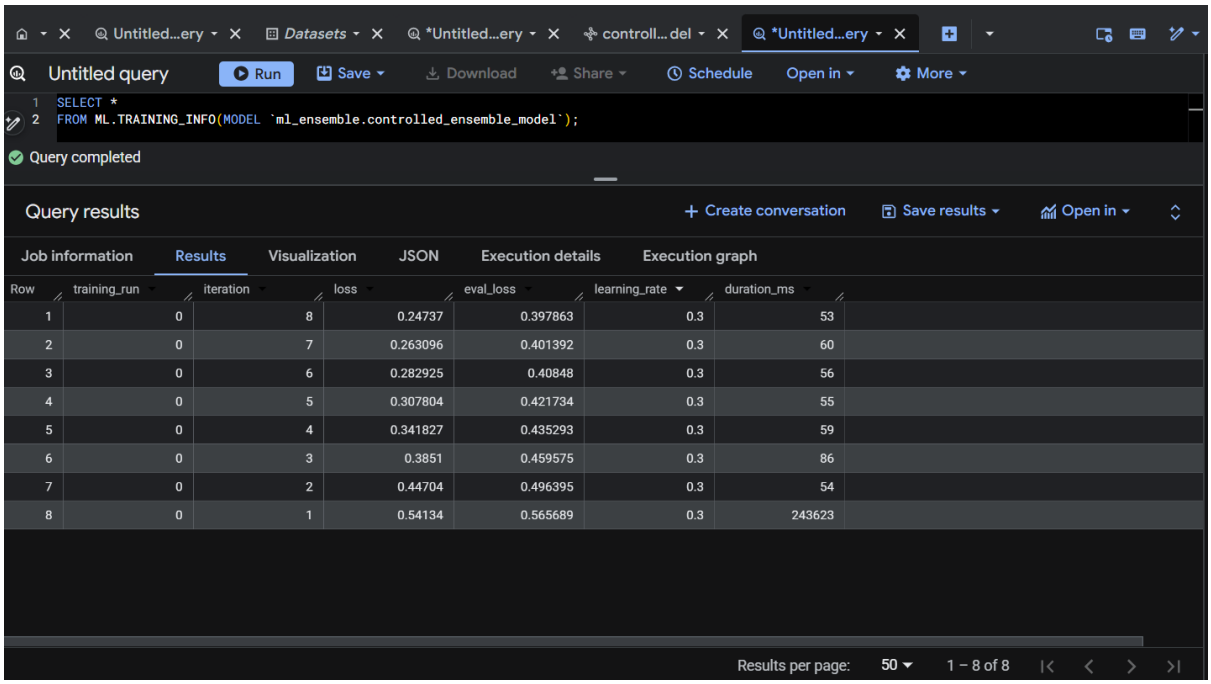
So, how do multiple models work together in ensemble learning? There are two main approaches. The first is bagging, where each model works independently. It is like an analyst panel or a courtroom panel where each judge works independently and gives a verdict. Each one has the same goal, but they work on their own, and then the final result is combined through voting or averaging, depending on the type of model.

The second approach is boosting. This is more like how learning happens step by step, where one stage focuses on the mistakes left by the earlier stage. The first model gives its result, and the next one focuses more on the parts where the earlier model struggled. Then the next one continues from there. So boosting is dependent in nature, because each later model builds on the weakness of the earlier one. This process continues until no further meaningful improvement is happening.

Now, moving to the implementation part, BigQuery ML supports the boosting method of ensemble, which I already knew because of the limitation I faced. I had used a max

depth of 3 without fully going into the basics of ensemble, because that stage was mainly to become familiar with tree behavior first. It was like opening a more complex tool but first understanding its basic structure. Now I am increasing the max depth to 5 to check whether that added depth helps reveal any stronger nonlinear signal in the dataset without immediately making the model too complex. If max depth 3 gives a shallow tree-style structure, then max depth 5 gives a more expanded one. So this step is meant to test whether a somewhat deeper ensemble structure captures something that the shallower version could not.

One early thing visible in the model behavior is that it stopped at 8 iterations, whereas with max depth 3 it had stopped at 11. This may suggest that after a certain point the model was not finding enough additional improvement to keep building further in the same way. But that alone is not the final answer. The more important question is still how the model behaves on the dataset and whether this added depth actually reveals stronger nonlinearity or not. That is what the next step should answer.



Sub Question 2 -

BigQuery ML only supports the boosting method for ensemble, so now the focus shifts directly to boosting, because that is the method I want to understand better. I already know that in boosting each model depends on the previous one and they work sequentially. But how does that actually improve prediction quality? What are the main things I should understand about how this method works? This becomes important because boosting is the ensemble structure I am using to analyze this dataset.

As I explained earlier, boosting is an ensemble method where multiple models are used together to predict results, but unlike bagging, these models do not work independently. In boosting, they work one after another. The main idea is that each later model focuses more on the mistakes left by the earlier one, and by gradually reducing those mistakes, the overall prediction error comes down. I could also observe this in the model with a maximum depth of 5, where the loss error and evaluation loss both kept reducing as the model moved forward. The main thing to understand here is gradual improvement. The model is not trying to solve everything in one attempt. It learns step by step, and that gradual correction itself helps create stability.

In simple terms, no one improves everything perfectly in one try. It normally takes repeated attempts. If everything is forced into the first step itself, the process can become too complex and may lead to overfitting. The same idea works here. Gradual learning helps the model improve its quality in a more controlled way. This is what was visible when the evaluation loss reduced over the iterations, showing that the model was learning step by step rather than trying to force everything at once.

The boosting method I am using here is gradient boosting. So now the important question is: if each later model focuses on the earlier mistakes, what kind of mistake is it trying to reduce? In simple terms, gradient boosting focuses on the remaining error left by the earlier stage and tries to improve that part gradually. So rather than treating the whole model as one fixed structure, it keeps correcting the parts that are still weak. That is why this method becomes useful when I want to check whether added complexity can capture something the earlier model could not.

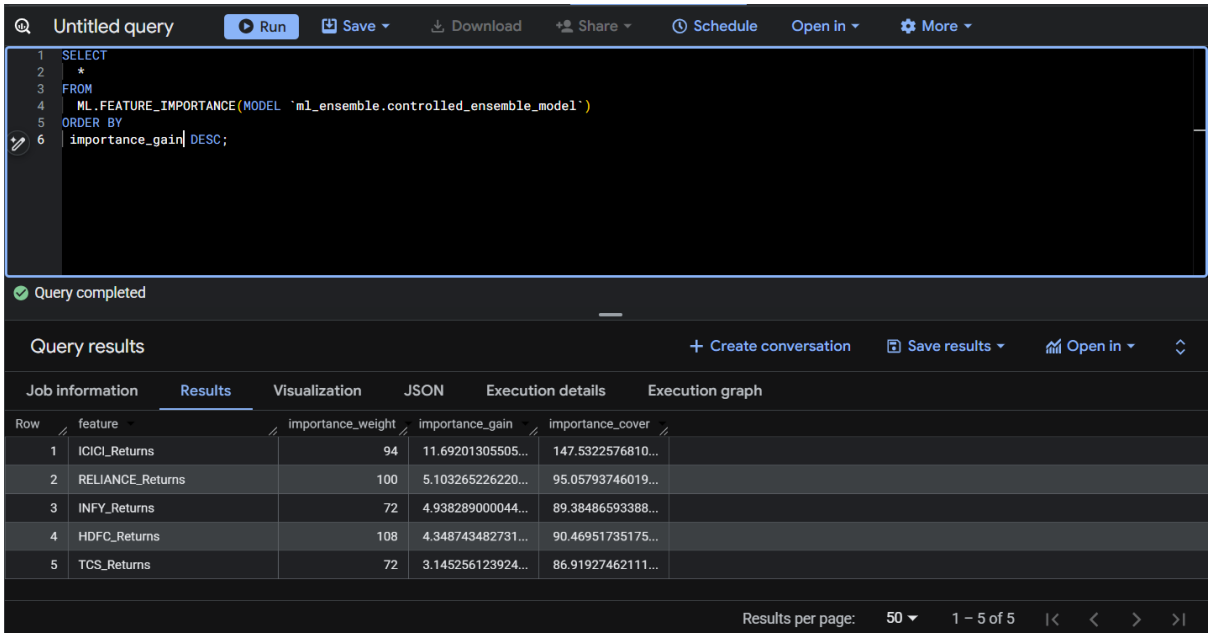
Looking at the model information table, one thing can be seen early: the model stopped at 8 iterations. This suggests that after that point it was not finding enough further improvement to continue meaningfully in the same direction. That is useful to notice, but it is not the final answer by itself. The more important thing is how the model behaves and whether this added boosting structure is actually revealing anything stronger in the dataset.

Another important part of this model is feature importance. I have discussed this before, but here it becomes important again, because understanding how the model makes its decisions matters a lot in this project. It is not only about the final prediction result. It is also about knowing which features were used, how often they were used, and whether the model is relying too much on one feature or splitting its logic in a more distributed way. That helps in understanding whether the model is stable and whether it is learning in a balanced way.

The feature importance results are somewhat similar to the earlier max-depth-3 model, but here the goal is to understand the effect more clearly under added depth. The

analysis shows that the HDFC feature was used more often than some others, but its contribution to reducing error or Gini impurity remains low. This means HDFC was used frequently, but it did not help the model as much as some other features. ICICI again appears much stronger in terms of effectiveness. Reliance was also used more here than in the max-depth-3 model, and although its effectiveness is somewhat lower than before, it still remains useful. One clear contrast appears between INFY and HDFC: HDFC is used much more, but its effectiveness is lower. So the feature importance result again shows a distributed structure, with no single feature fully dominating, but some features still being much more effective than others. In that sense, ICICI again appears to be the strongest feature in helping the model reduce confusion and improve the split quality.

Overall, the feature importance shows that the model is using multiple features rather than depending on only one. It also shows that the model is trying to combine smaller signals from different features to improve prediction quality step by step. But whether this added complexity is truly revealing stronger nonlinearity or not still needs one more check. For that, the next step is to train an unrestricted model and see whether the current depth restriction is limiting the nonlinear signal, or whether the dataset itself is mostly linear in its directional structure.



Sub Question 3 -

After analyzing the model behavior and feature importance of the restricted ensemble model with depth 5, the results started leaning toward the idea that the directional signal is mostly linear. To check that more properly, I also used an unrestricted model. The purpose of this was to see whether the unrestricted model could capture any

stronger nonlinear pattern that the restricted model failed to capture, or whether it would mainly start moving toward overfitting.

So the flow of this question works in this way. First, I compare the model information and feature importance of both models. Then, using the prediction results, I compare the train and test performance of both and try to understand whether any meaningful nonlinearity exists or whether the structure is still mostly linear.

Before moving into the result, understanding the bias-variance idea becomes important here. Even though both of these are ensemble models, the basic logic still matters. If a model is too simple and cannot even capture the main theme in the training data, that suggests higher bias. But if a model becomes too complex and starts trying to capture too many individual points, then it moves toward higher variance. In that case, the training result may improve, but the test result may weaken, because the model is learning more noise than signal. So the main aim is not just to make the model more complex, but to see whether that extra complexity improves the real fit or only increases noise capture.

Now, looking at the unrestricted model, one early thing visible is that the iteration count is again 8. Also, the deepest tree goes to a depth of 6. This means the unrestricted model is clearly more complex than the restricted one, but at the same time it still does not keep expanding endlessly. The loss and evaluation loss continue to show a gradual decline, which suggests that the model remains internally stable during training. So from the model table alone, the unrestricted model does not immediately look like a collapsed structure.

However, feature importance gives a more useful story. In this model, HDFC is no longer the main feature being used most heavily. Instead, the model leans more toward Reliance, but that increased use does not translate into equally strong effectiveness. In fact, both Reliance and HDFC show weaker effectiveness compared with ICICI. Just like in the earlier models, all features are still being used in a distributed way, and no single feature dominates the full structure. But across all the tree-based versions, one pattern stays consistent: ICICI remains the most effective feature, even when it is not the most used one. That is an important repeated signal across the models.

From this comparison alone, there is still not enough evidence to say that a strong nonlinear pattern has appeared. The unrestricted model is more complex, but the internal structure still does not show a dramatic shift in the nature of the signal. A clearer answer comes when I move from the model table to the prediction results.

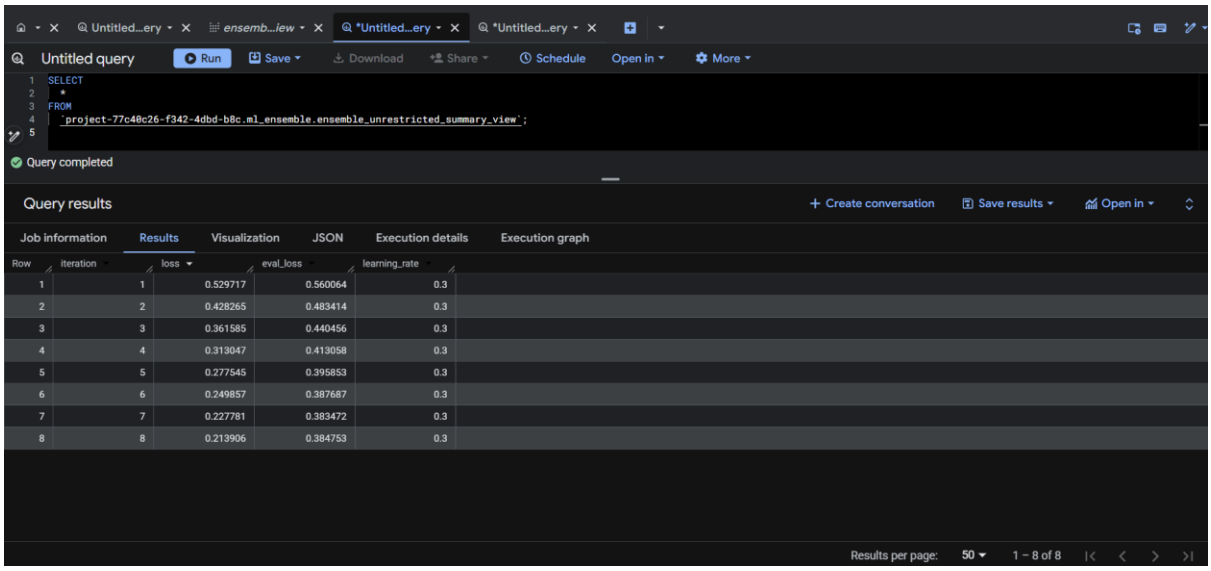
Another point linked with overfitting also becomes relevant here. In overfitting, the model usually learns the training data too well, but then fails to generalize on the test data. In the internal model evaluation, both loss and evaluation loss were decreasing,

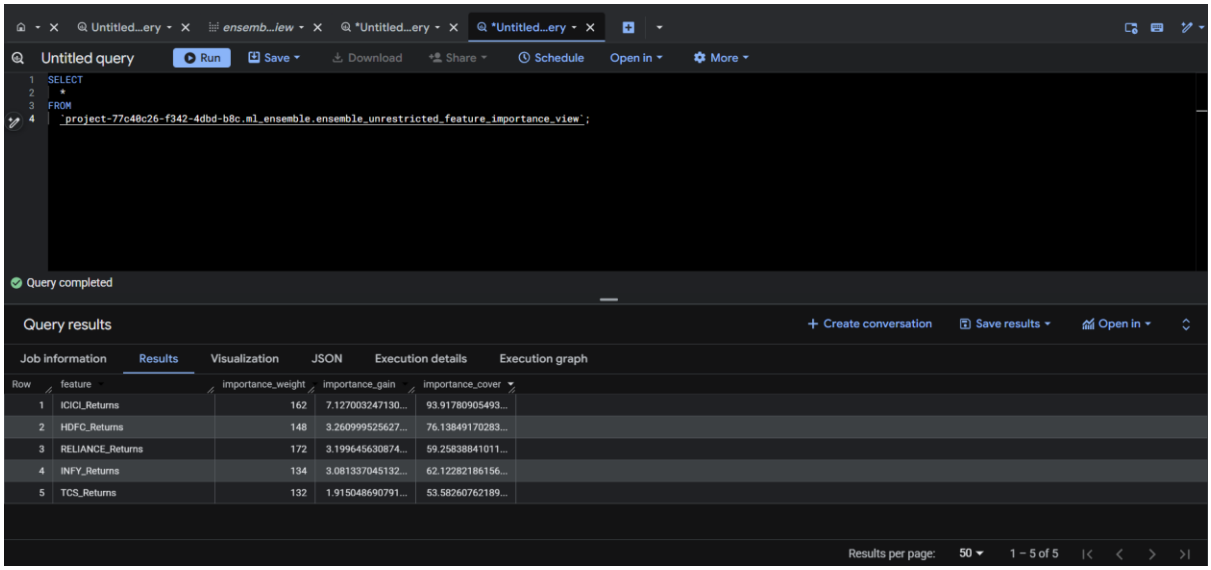
so the model does not immediately look unstable from that internal view. But the clearer answer comes from comparing train and test prediction performance directly.

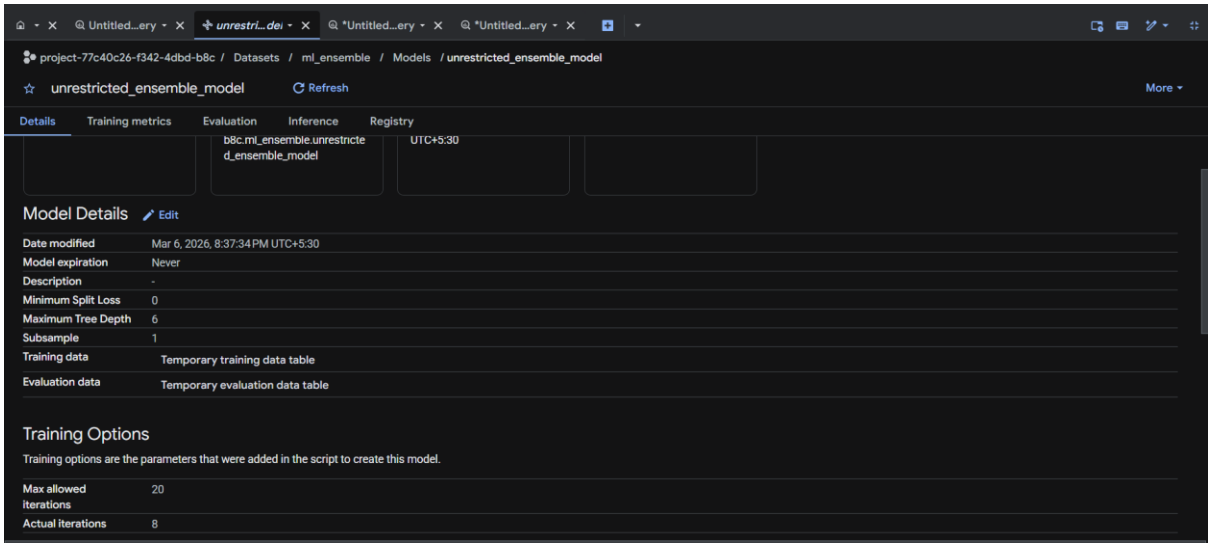
The accuracy results of the controlled and unrestricted models show the main pattern. Both models perform very well on the training data. The controlled model reaches nearly 90% training accuracy, and the unrestricted model goes even higher, above 90%. But on the test data, the pattern goes in the other direction. The controlled model falls to around 83%, and the unrestricted model falls slightly more, to around 82.58%. So as model complexity increases, training accuracy improves, but test accuracy weakens. This widening gap suggests that the added complexity is helping the model fit the training data more closely, but not helping it generalize better to unseen data.

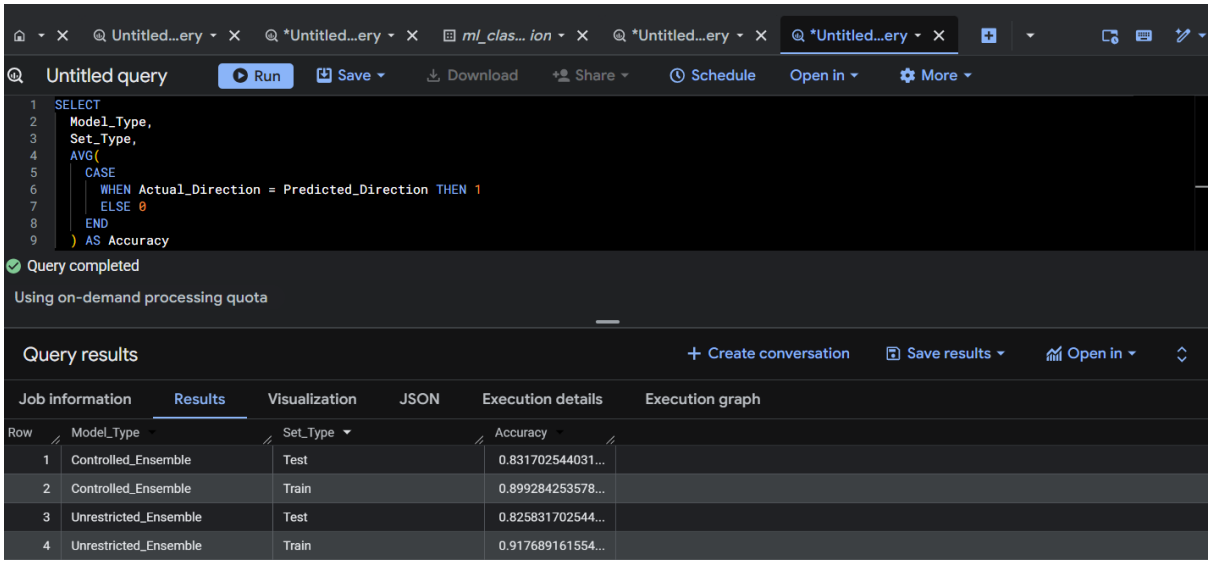
This is a very important pattern. It suggests that once the model moves beyond a certain level of depth, it starts capturing more noise from the training data rather than discovering a stronger underlying nonlinear signal. That is why the training score improves while the test score does not. At the same time, the test accuracy does not collapse completely, so the model cannot be called unstable. But it also cannot be called more useful just because the training score became higher.

So, comparing the restricted and unrestricted models, the overall pattern suggests that the directional signal is still mostly linear in nature. More complexity is increasing fit on the training data, but it is not improving the test result. This means the added depth is giving more noise capture than real signal improvement. The next step now is to understand these models further through precision, recall, and the confusion matrix, so it becomes clearer whether there is any hidden bias in the way these predictions are being made.




Untitled query
Run
Save
Download
Share
Schedule
Open in
More
SELECT
*
FROM
`project-77c40c26-f342-4dbd-b8c.ml_ensemble.ensemble_unrestricted_feature_importance_view`;
Query completed
Query results
+ Create conversation
Save results
Open in
Job information
Results
Visualization
JSON
Execution details
Execution graph
Row feature importance_weight importance_gain importance_cover
1 ICICI_Returns 162 7.127003247130... 93.91780905493...
2 HDFC_Returns 148 3.260999525627... 76.13849170283...
3 RELIANCE_Returns 172 3.199645630874... 59.25838841011...
4 INFY_Returns 134 3.081337045132... 62.12282186156...
5 TCS_Returns 132 1.915048690791... 53.58260762189...
Results per page: 50
1 – 5 of 5


project-77c40c26-f342-4dbd-b8c / Datasets / ml_ensemble / Models / unrestricted_ensemble_model
unrestricted_ensemble_model
Refresh
More
Details
Training metrics
Evaluation
Inference
Registry
b8c.ml_ensemble.unrestricte
d_ensemble_model
UTC+5:30
Model Details
Edit
Date modified Mar 6, 2026, 8:37:34 PM UTC+5:30
Model expiration Never
Description -
Minimum Split Loss 0
Maximum Tree Depth 6
Subsample 1
Training data Temporary training data table
Evaluation data Temporary evaluation data table
Training Options
Training options are the parameters that were added in the script to create this model.
Max allowed iterations 20
Actual iterations 8


Untitled query
Run
Save
Download
Share
Schedule
Open in
More
SELECT
Model_Type,
Set_Type,
AVG(
CASE
WHEN Actual_Direction = Predicted_Direction THEN 1
ELSE 0
END
) AS Accuracy
Query completed
Using on-demand processing quota
Query results
+ Create conversation
Save results
Open in
Job information
Results
Visualization
JSON
Execution details
Execution graph
Row Model_Type Set_Type Accuracy
1 Controlled_Ensemble Test 0.831702544031...
2 Controlled_Ensemble Train 0.899284253578...
3 Unrestricted_Ensemble Test 0.825831702544...
4 Unrestricted_Ensemble Train 0.917689161554...

Sub Question 4 -

After analyzing the gap between train and test accuracy in these two models, my instinct still favors the idea that the directional signal is mostly linear in nature. But the main goal here is to understand these two models better, compare them with the previous models, and finally judge whether any strong nonlinear pattern really exists or not. This part brings together the results from all the models I have evaluated so far, because this final comparison should tell me whether the added complexity actually gives anything more useful. One thing is already clear: the ICICI feature has remained the most effective feature across the tree-based models, which shows some consistency and stability in how these models behave. But stability alone is not the final goal. Usefulness is. The real question is whether the directional signal of this dataset is simple in nature or not.

One of the most important things I already discussed is the increase in the gap between train and test accuracy as model complexity increases. The gap is still small in absolute terms, and each model's test accuracy remains above 80%, so there is no collapse. But the important pattern is that as train accuracy keeps increasing, test accuracy starts decreasing. This shows that the training model is becoming more complex and is learning more from the training data, but that added learning is not being reflected in the test results. That is the main concern. The models remain stable, but the extra complexity is not giving extra usefulness. This again suggests that the signal may already be captured by a broader rule, rather than by deeper conditional sub-rules.

So the next step is to understand the remaining metrics such as precision, recall, and the confusion matrix for these two models, and then compare them with the earlier models as well. In total, this gives a four-model comparison, which helps in judging the full structure better.

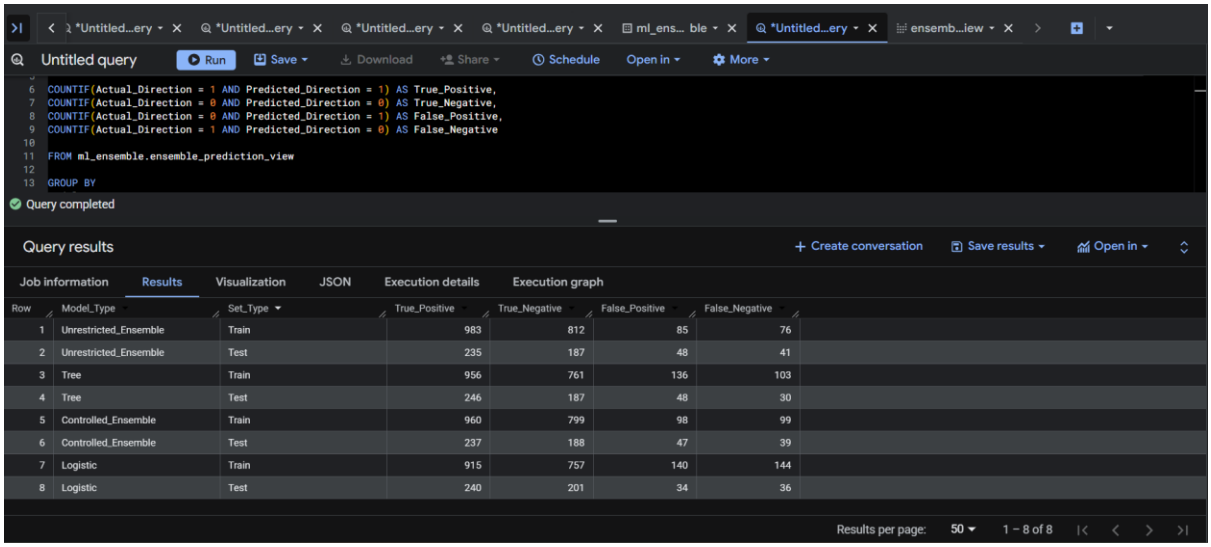
The confusion matrix across all four models shows something important. I focus first on the false positives because they are easier to compare clearly across the models. In the training data, the false positive count in the logistic model is higher than in the tree-based models, and the lowest value appears in the unrestricted model. This helps explain why the unrestricted model shows such high training accuracy. But when I compare the same pattern in the test data, the gap does not remain that strong. The test-side false positives are much closer across the models, and the same broad pattern is visible in the true positives as well. This supports the point already discussed: the more complex model is learning extra things in the training data, but those extra gains are not being justified in the test data.

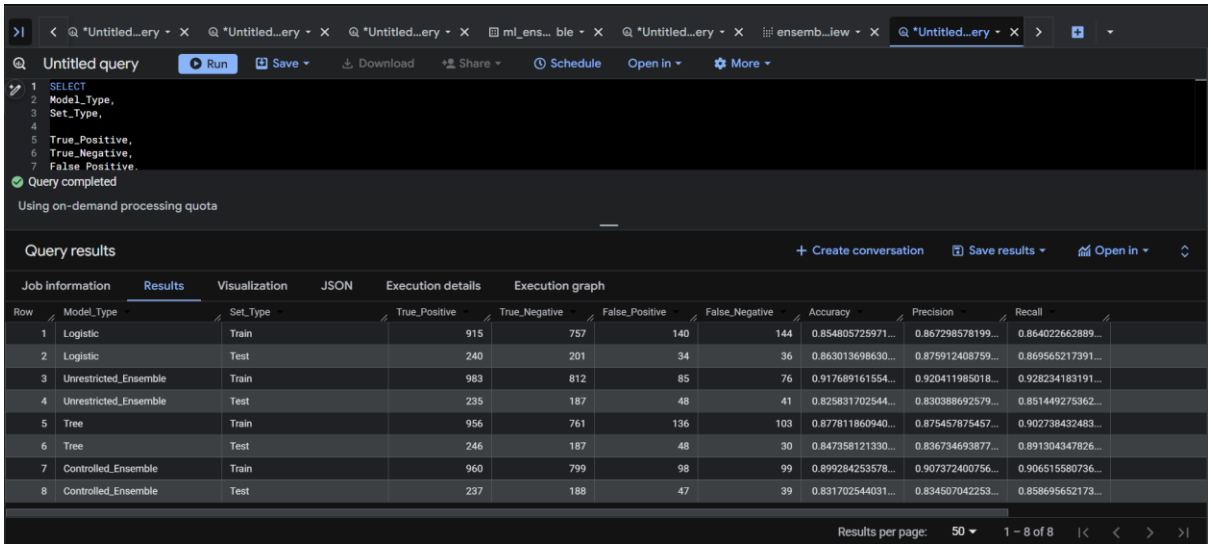
Another useful comparison is the ratio of false positives to true positives. In the unrestricted model, that ratio stays below 10% in the training data, but rises above 20%

in the test data. This again suggests that the model is learning some extra pattern from the training set that does not carry properly into the test set. That is a sign of noise capture, or at least a sign that extra complexity is not translating into better generalization.

The same pattern appears in precision, recall, and accuracy. The increase in train metrics does not lead to stronger test metrics. In fact, after a point, the test metrics begin to weaken slightly as complexity increases. So when the metrics table is viewed as a whole, it shows that all the models are stable, but added complexity is not creating any new benefit. Most of the test results remain similar, and in some cases they even decline slightly as complexity increases. That means more complex models are not adding much more useful understanding to this dataset.

Overall, it can be concluded that the usefulness of the more complex models is limited, and they show signs of overfitting. At the same time, all the models remain stable because none of them collapse on the test data. But adding more complexity does not help much in this dataset. So the directional signal is likely mostly linear in nature, and that broader signal is already being captured well by the logistic regression model, or the classification model.





Answer to the fourth major question -

The fourth major question was about whether bringing in the ensemble tree more directly, with greater depth and complexity, would reveal any stronger non-linear pattern in the dataset, or whether it would mainly move toward overfitting. Unlike the previous stage, where I used a shallow tree-style structure to understand decision-tree behavior first, this stage focused more clearly on the ensemble tree for what it actually is in BigQuery ML. Since BigQuery ML does not support a standalone single decision tree in the way I wanted to test it, this question became important to understand whether added ensemble complexity itself could reveal something more useful in the dataset.

The analysis shows a fairly clear result. Both ensemble models used in this stage remained stable in nature, because the test accuracy did not collapse and the overall metrics stayed above a good level. So the models cannot be called failures. However, at the same time, they also showed signs of overfitting. As the complexity of the model increased, the training accuracy improved, but the test accuracy weakened slightly, and the gap between train and test accuracy became wider. This pattern suggests that the added complexity was helping the model fit the training data more closely, but that extra fit was not turning into better generalization on the test data.

This is the main point. If the deeper ensemble structure had captured some strong hidden non-linear pattern, then that added complexity should have improved the test result or at least provided some new usefulness compared with the simpler classification model. But that did not happen. Instead, the model became better at fitting the training data while becoming slightly weaker on the test data. This suggests that the extra complexity was mostly capturing noise from the training dataset rather than discovering a stronger underlying nonlinear signal.

When compared with the earlier models, the picture becomes even clearer. The classification model had already captured the directional signal of the dataset well, and the later ensemble models did not extend that usefulness in any meaningful way. Their test accuracy remained in a generally good range, but they did not improve the result and in some cases slightly lowered it. So the added complexity did not produce added value.

The main takeaway from this question is that there is no strong evidence of a hidden nonlinear pattern in the dataset that becomes more useful when ensemble complexity is increased. The directional signal still appears to be mostly linear in nature, and that broader signal was already captured well by the classification model. So for this dataset, the usefulness of the more complex ensemble model remains limited, because it adds complexity without adding enough new understanding.

The Fifth Major Question of the Project -

After analyzing all the models, from regression to the unrestricted ensemble models, some points have become clear. As model complexity increases, the training accuracy increases, but the testing accuracy tends to decrease, and the gap between train and test also becomes wider. At the same time, none of the models collapse on the test data, so overall they remain stable in nature. When I compare the test accuracies, the difference across the models is also not very large. Most of them remain above 80%, and in many cases the difference is only around 1–2%.

Another thing that becomes visible is that adding more complexity or more models does not automatically improve the usefulness of the result for this dataset. This raises an important question. Can I simply conclude that the logistic model is the best only by looking at accuracy? Is that enough to make the final judgment? Or is the situation more complex than that?

Simplicity does matter, because if more complex models are not adding any new usefulness, then the simpler model naturally becomes stronger in practical terms. But at the same time, if the accuracy differences are small and most models remain stable, then accuracy alone may not be enough to declare one model clearly superior. The ensemble models do show signs of overfitting, but they still do not collapse, because the training process stops before that kind of full failure happens. So the final question now is this: what other ways should these models be evaluated so that I can build better evidence and make a more reliable conclusion about which model is actually better for this dataset?

Sub Question 1 -

The first question that arises when analyzing these models further is whether accuracy alone is enough to judge them properly. Even though all the models are stable, each one has its own way of predicting and understanding the dataset. So relying only on accuracy is not a good decision. The main issue is that even when the accuracy scores are similar, the models may be behaving very differently internally. This may be because of complexity, model structure, or the way they assign probability before giving the final category. So the key question is whether accuracy alone can decide which model should be chosen, or whether there are limitations in accuracy that it cannot reveal.

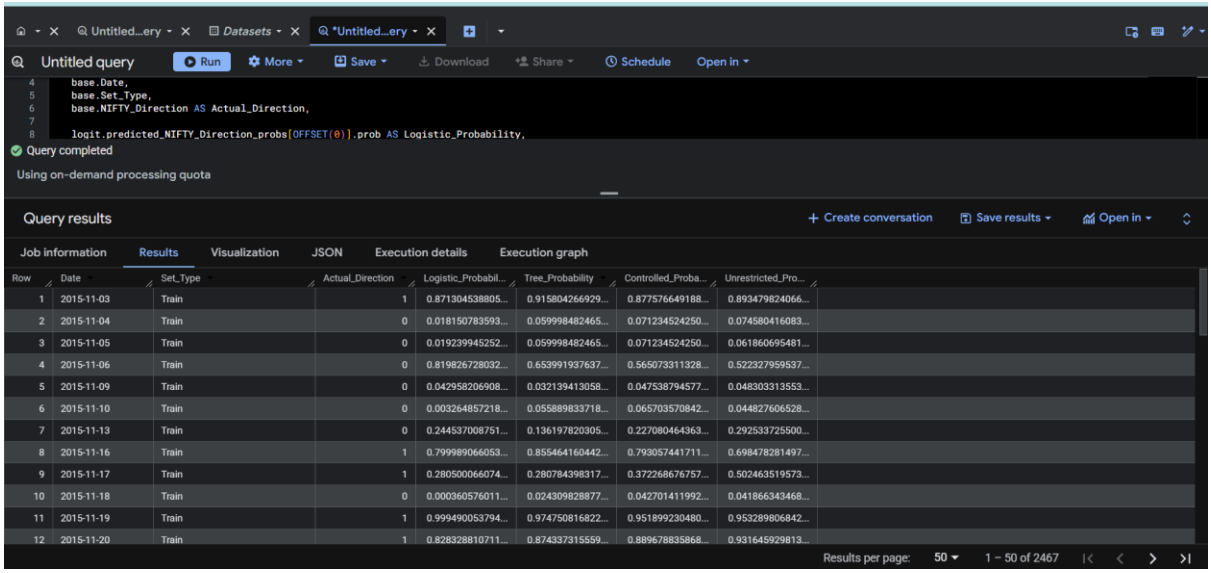
One important limitation of accuracy becomes clear when I understand how a classification model actually works. A classification model gives a final category, but before that it first predicts a probability. In binary classification, that probability is usually for one default class, which in this dataset is the “Up” class. Since I am using logistic regression as one of the models, the model first predicts the probability of “Up,” and then that probability is converted into a final category using the default threshold of 0.50. If the probability is below 0.50, the prediction becomes “Down,” and if it is 0.50 or above, it becomes “Up.”

Accuracy only checks whether the final predicted category is correct or not. It does not care how confident the model was when making that prediction. That is the important limitation. For example, if one row has an “Up” probability of 51% and another has an “Up” probability of 91%, both will be classified as “Up,” and accuracy will treat both in the same way. But in reality, those two predictions are not equally strong. One is only slightly above the threshold, while the other is much more confident. So the probability itself matters more than only the final category when I want to compare models more deeply.

Another limitation is that accuracy can change if the classification threshold is changed, even though the predicted probabilities themselves remain the same. That means accuracy depends not only on the model, but also on the threshold rule used to convert probability into category. Because of this, accuracy alone cannot fully show which model is stronger. Understanding the predicted probabilities from different models can help me judge which model gives more reliable and more meaningful confidence in its predictions.

So, for this analysis, I need to create a dataset that includes the predicted probabilities from each model. This will allow me to compare them properly and later apply other evaluation methods that work directly on probabilities rather than only on final categories. In BigQuery ML, the prediction output gives the probabilities for both “Up” and “Down,” but I am focusing on the “Up” probability, which is the default class in this

case. I will take the “Up” probability from each model for each data point and combine them into one comparison dataset for further analysis.



Sub Question 2 -

Now, after understanding the limitations of the accuracy metric, we see that the limitation indicates that accuracy focuses on the threshold of the prediction direction. This can change if I adjust the threshold, but the main function of the model is to predict probabilities, which reveal how the model performs over the dataset.

First, I extract the probabilities from each model and create a dataset. However, a new question or curiosity arises: what should I do with this probability dataset? I can't analyze all 2467 rows like that, and it shouldn't be necessary. I need to understand how each model performs- whether they are confident enough in their predicted probabilities. So, how should I analyze the data before applying any specific metric? How can I scan it properly to get my initial impression of how each model analyzes the dataset and their confidence levels about the predictions?

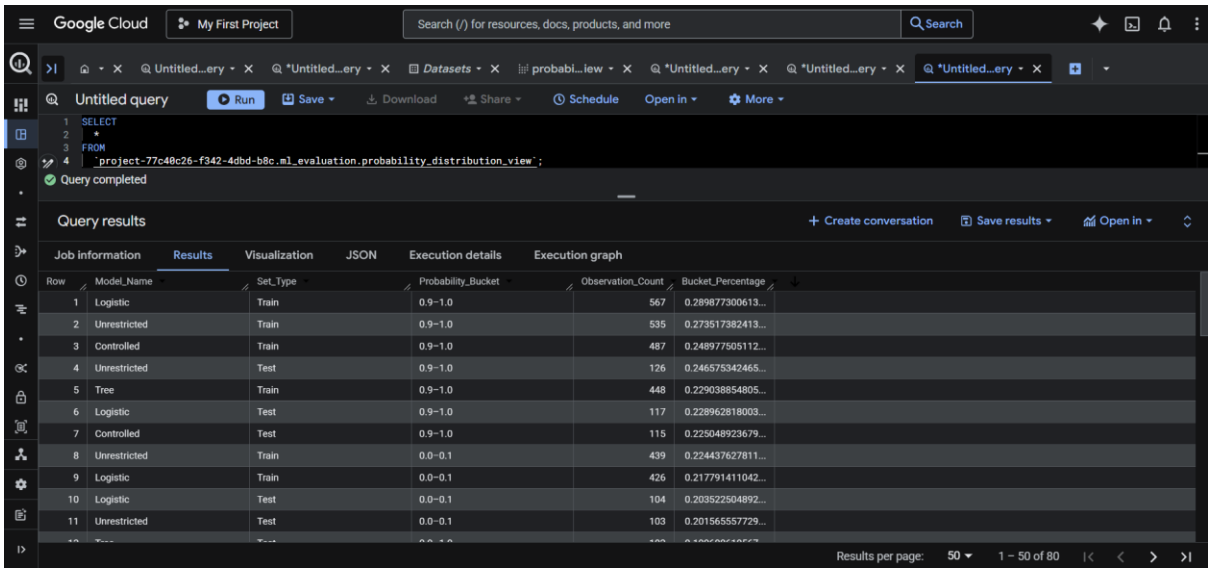
The answer to how to analyze these probabilities firsthand is through the probability distribution. Probability distribution is something I have done often in other ways, such as making ranges of scores for many students, putting them on a bar chart, analyzing these score ranges, and understanding how many students score above 80 or in other categories. The same approach can be applied here by grouping these probabilities into ranges or buckets, which I can label as above 90% (highly confident), 50-60%, 40-50% (more uncertain), etc. This allows me to analyze the probabilities better and gain initial insights before diving deeper later on. Additionally, I can calculate the percentage share for each range and then create broader labels to assess whether the model is confident most of the time or uncertain—for example, combining ranges like 70-100% and 0-30% to show confidence, and 30-70% to indicate uncertainty. I can also create other broader

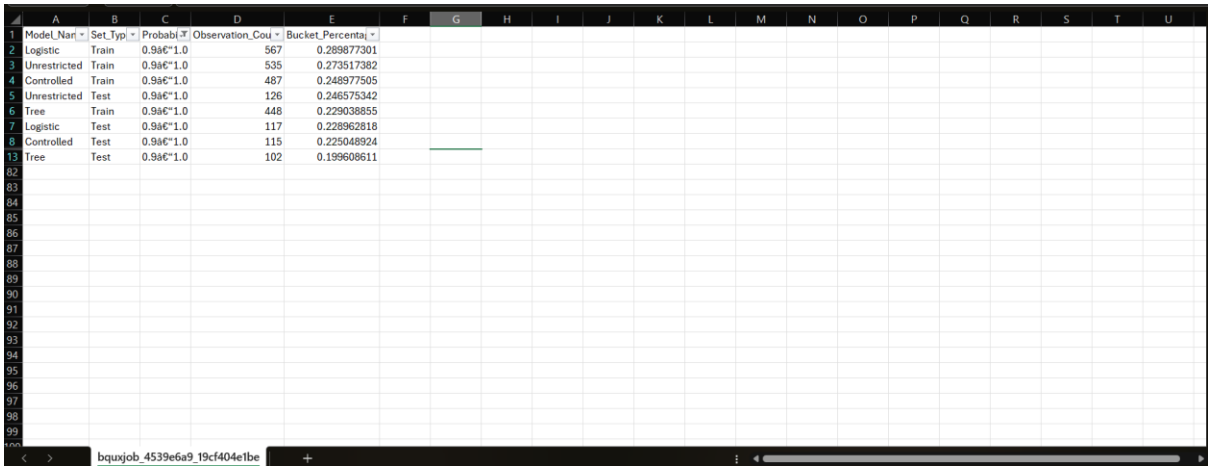
labels with different ranges. These are my thoughts before actually implementing this on the probability dataset, helping me understand what steps to take once I have the probability distribution.

Now, one thing to understand, since I have never done this before, is how I perform the query. This is important because I believe the probability view is in the wide format, but for distribution in the ranges, we need to use GROUP BY in SQL. First, we have to convert the dataset into a long format so we can easily group it by model type. Then, I created the probability buckets and aggregated the data, getting each model type and probability bucket as train and test datasets.

The view of this is very tough to analyze overall, but I will focus on the first impression. It's hard to judge which model performed better based solely on the probability distribution. However, by filtering in Excel, I noticed that the logistic and unrestricted models had the highest confidence percentages in the 90-100 range, despite the test percentage dropping significantly compared to training. It's not easy to determine which model is better, but the pattern shows that, as we go from 100 to 50, the percentage naturally drops, and the same happens from 0 to 50. Still, it's very difficult to interpret this probability distribution to draw any concrete conclusions. This leads to another question: which metrics can better help us understand these probability predictions? Since the probability distribution is too opaque for me, and even though I created many buckets, comparing them side by side remains very challenging.

Overall, using the probability distribution for analysis becomes difficult for me because it is too opaque, and the percentages are very close to each other. Additionally, the many probability buckets create another challenge in understanding. So, the next question arises: is there any metric that can precisely give the value of probabilities, similar to how we have accuracy, precision, and recall for threshold values or not?





	A	B	C	D	E
1	Model_Nar	Set_Typ	Probabi	Observation_Cou	Bucket_Percenta
2	Logistic	Train	0.9â€"1.0	567	0.289877301
3	Unrestricted	Train	0.9â€"1.0	535	0.273517382
4	Controlled	Train	0.9â€"1.0	487	0.248977505
5	Unrestricted	Test	0.9â€"1.0	126	0.246575342
6	Tree	Train	0.9â€"1.0	448	0.229038855
7	Logistic	Test	0.9â€"1.0	117	0.228962818
8	Controlled	Test	0.9â€"1.0	115	0.225048924
13	Tree	Test	0.9â€"1.0	102	0.199608611

bquxjob_4539e6a9_19cf404e1be

Sub Question 3 -

The main question that arises after trying to analyze the probability distribution through buckets is whether there is any probability-based metric that can give a clearer value to help decide which model is better at probability prediction.

The answer to this question comes through the ROC curve. The ROC curve directly addresses one of the biggest limitations of accuracy, which is that accuracy changes when the threshold changes. A classification model gives probabilities first, and then a threshold is applied to turn those probabilities into final labels. So if the threshold changes, the final accuracy can also change, even when the predicted probabilities themselves remain the same. The ROC curve deals with this issue by checking model behavior across many thresholds instead of depending on only one.

So how does the ROC curve work? It is built using two values: TPR and FPR. TPR is the true positive rate, which is the same as recall in this setting. It shows how many actual positives the model correctly identifies. FPR is the false positive rate, which shows how often the model incorrectly labels negative cases as positive. So ROC is basically checking the trade-off between correctly identifying positives and incorrectly pushing negatives into the positive side.

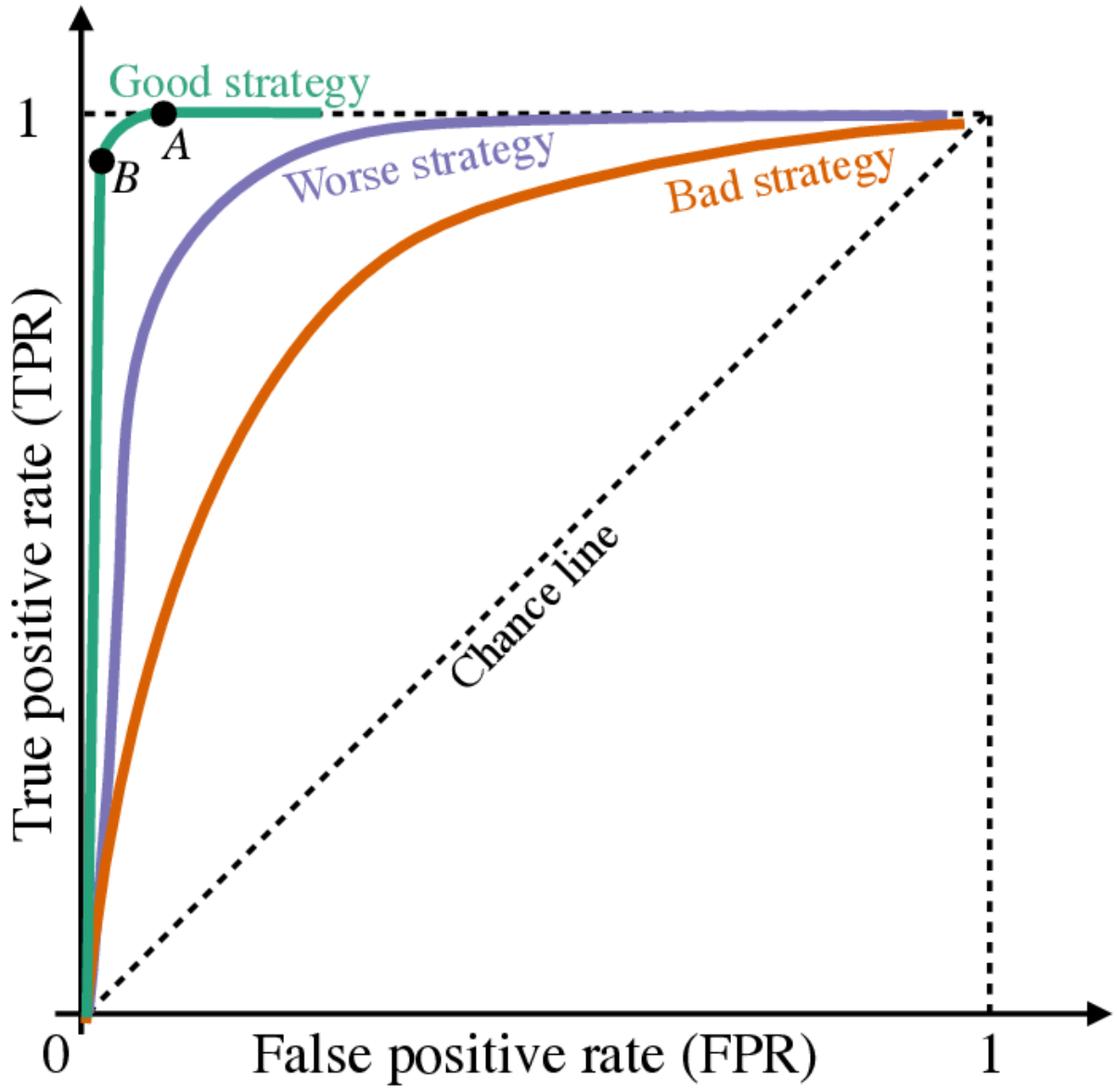
In the ROC curve, FPR is placed on the x-axis and TPR on the y-axis. The purpose of the curve is to show how well the model separates positive and negative outcomes across different threshold values. If the model is strong, then for many thresholds it should keep TPR high while keeping FPR low. That is why the top-left side of the ROC space is considered better. A model closer to that side is better at separating positive and negative cases. If the curve stays closer to the middle, it means the model is not separating them very confidently.

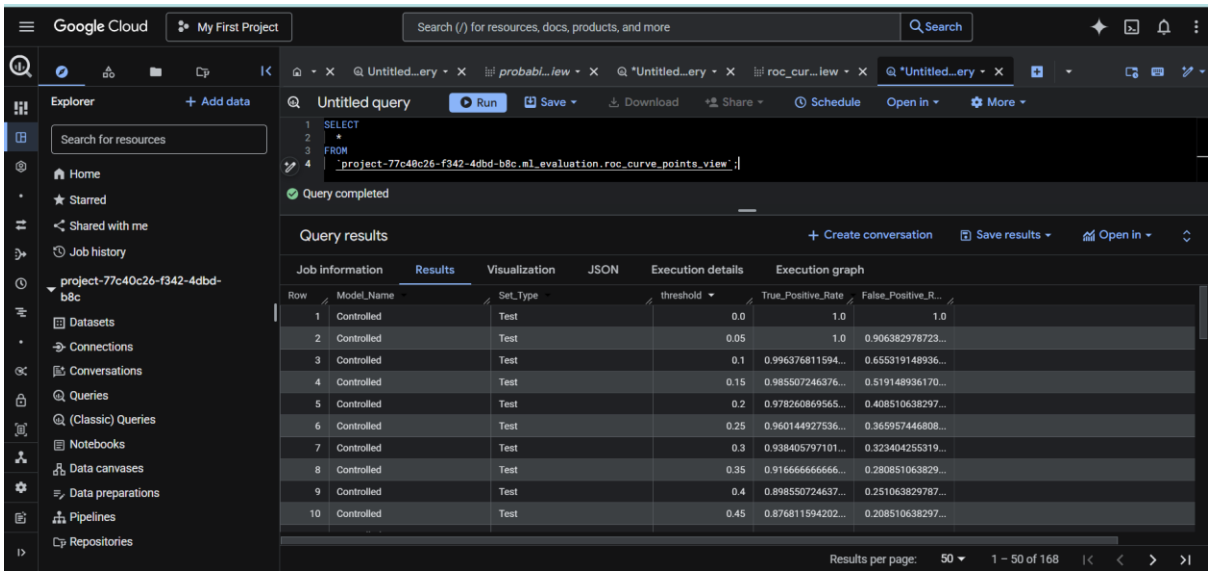
For implementation, I again converted the wide table into a long format so I could group results by model type more easily. Then I created thresholds from 0.0 to 1.0 in steps of

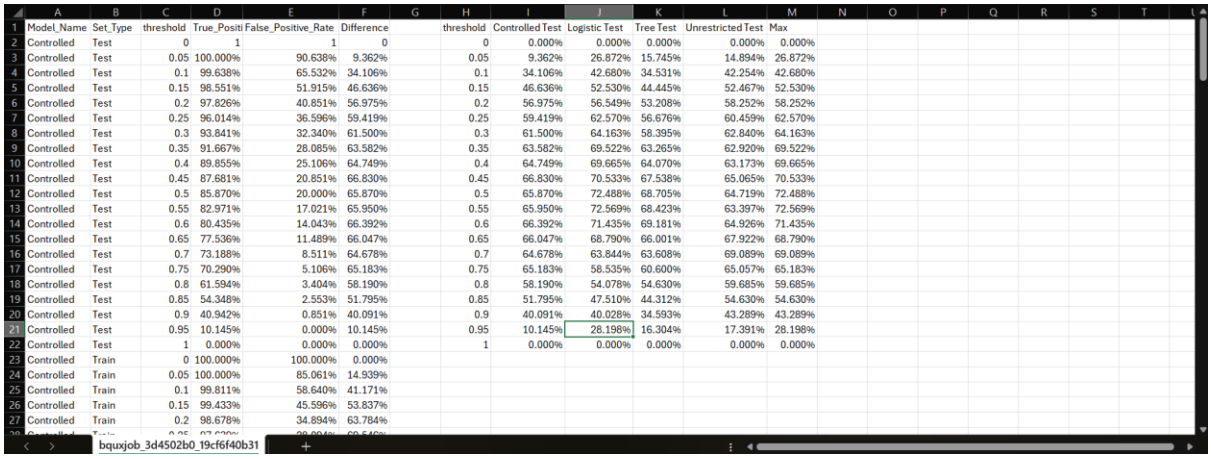
0.05, giving 21 thresholds in total. For each threshold, I created the predicted label, built the confusion matrix, and then calculated TPR and FPR. In the end, I created a result table organized by threshold, set type, and model type, along with the corresponding TPR and FPR values.

Now, analyzing the result table was not very simple, just like the probability distribution was not simple. But I used one practical way to read it: I checked which model showed larger differences between TPR and FPR, especially in the test data, because the test result matters more. From this comparison, the logistic model showed the strongest separation between TPR and FPR overall, and the unrestricted model came after that. One example that stood out was the 95% threshold, where the FPR for all models was 0%, but the logistic model still had a higher TPR than the others. This shows that even at a very strict threshold, the logistic model was still identifying more true positives without increasing false positives. That is a strong sign of confidence. But at the same time, I also understand that ROC is not only about one threshold. It is about how the model behaves across all thresholds.

So overall, the ROC analysis gave useful insight. It showed that the logistic model separates positive and negative outcomes better than the other models across thresholds, with the unrestricted model appearing next best in some parts. But the main difficulty still remains: ROC gives many values across many thresholds, so it is still not easy to summarize into one clear overall judgment in the way accuracy does. That naturally leads to the next question: is there any single value that can summarize the overall ROC behavior and help judge the probability quality of the models more clearly?


Good strategy
Worse strategy
Bad strategy
Chance line
A
B
1
0
1
True positive rate (TPR)
False positive rate (FPR)


Google Cloud
My First Project
Search (/) for resources, docs, products, and more
Search
Explorer
Add data
Search for resources
Home
Starred
Shared with me
Job history
project-77c40c26-f342-4dbd-b8c
Datasets
Connections
Conversations
Queries
(Classic) Queries
Notebooks
Data canvases
Data preparations
Pipelines
Repositories
Untitled query
Run
Save
Download
Share
Schedule
Open in
More
SELECT
*
FROM
`project-77c40c26-f342-4dbd-b8c.ml_evaluation.roc_curve_points_view`;
Query completed
Query results
Create conversation
Save results
Open in
Job information
Results
Visualization
JSON
Execution details
Execution graph
Row
Model_Name
Set_Type
threshold
True_Positive_Rate
False_Positive_R...
1 Controlled Test 0.0 1.0 1.0
2 Controlled Test 0.05 1.0 0.906382978723...
3 Controlled Test 0.1 0.99637681159... 0.655319148936...
4 Controlled Test 0.15 0.985507246376... 0.519148936170...
5 Controlled Test 0.2 0.978260869565... 0.408510638297...
6 Controlled Test 0.25 0.960144927536... 0.365957446808...
7 Controlled Test 0.3 0.938405797101... 0.323404255319...
8 Controlled Test 0.35 0.916666666666... 0.280851063829...
9 Controlled Test 0.4 0.898550724637... 0.251063829787...
10 Controlled Test 0.45 0.876811594202... 0.208510638297...
Results per page: 50
1 – 50 of 168



Model_Name	Set_Type	threshold	True_Positi	False_Positive_Rate	Difference		threshold	Controlled Test	Logistic Test	Tree Test	Unrestricted Test	Max
Controlled	Test	0	1	1	0		0	0.000%	0.000%	0.000%	0.000%	0.000%
Controlled	Test	0.05	100.000%	90.638%	9.362%		0.05	9.362%	26.872%	15.745%	14.894%	26.872%
Controlled	Test	0.1	99.638%	65.532%	34.106%		0.1	34.106%	42.680%	34.531%	42.254%	42.680%
Controlled	Test	0.15	98.551%	51.915%	46.636%		0.15	46.636%	52.530%	44.445%	52.467%	52.530%
Controlled	Test	0.2	97.826%	40.851%	56.975%		0.2	56.975%	56.549%	53.208%	58.252%	58.252%
Controlled	Test	0.25	96.014%	36.596%	59.419%		0.25	59.419%	62.570%	56.676%	60.459%	62.570%
Controlled	Test	0.3	93.841%	32.340%	61.500%		0.3	61.500%	64.163%	58.395%	62.840%	64.163%
Controlled	Test	0.35	91.667%	28.085%	63.582%		0.35	63.582%	69.522%	63.265%	62.920%	69.522%
Controlled	Test	0.4	89.855%	25.106%	64.749%		0.4	64.749%	69.665%	64.070%	63.173%	69.665%
Controlled	Test	0.45	87.681%	20.851%	66.830%		0.45	66.830%	70.533%	67.538%	65.065%	70.533%
Controlled	Test	0.5	85.870%	20.000%	65.870%		0.5	65.870%	72.488%	68.705%	64.719%	72.488%
Controlled	Test	0.55	82.971%	17.021%	65.950%		0.55	65.950%	72.569%	68.423%	63.397%	72.569%
Controlled	Test	0.6	80.435%	14.043%	66.392%		0.6	66.392%	71.435%	69.181%	64.926%	71.435%
Controlled	Test	0.65	77.536%	11.489%	66.047%		0.65	66.047%	68.790%	66.001%	67.922%	68.790%
Controlled	Test	0.7	73.188%	8.511%	64.678%		0.7	64.678%	63.844%	63.608%	69.089%	69.089%
Controlled	Test	0.75	70.290%	5.106%	65.183%		0.75	65.183%	58.535%	60.600%	65.057%	65.183%
Controlled	Test	0.8	61.594%	3.404%	58.190%		0.8	58.190%	54.078%	54.630%	59.685%	59.685%
Controlled	Test	0.85	54.348%	2.553%	51.795%		0.85	51.795%	47.510%	44.312%	54.630%	54.630%
Controlled	Test	0.9	40.942%	0.851%	40.091%		0.9	40.091%	40.028%	34.593%	43.289%	43.289%
Controlled	Test	0.95	10.145%	0.000%	10.145%		0.95	10.145%	28.198%	16.304%	17.391%	28.198%
Controlled	Test	1	0.000%	0.000%	0.000%		1	0.000%	0.000%	0.000%	0.000%	0.000%
Controlled	Train	0	100.000%	100.000%	0.000%							
Controlled	Train	0.05	100.000%	85.061%	14.939%							
Controlled	Train	0.1	99.811%	58.640%	41.171%							
Controlled	Train	0.15	99.433%	45.596%	53.837%							
Controlled	Train	0.2	98.678%	34.894%	63.784%							

bquxjob_3d4502b0_19cf6f40b31

Sub Question 4 -

This evaluation question has been very difficult to answer so far. First, I moved away from accuracy as the main metric for comparing models, because accuracy can change when the threshold changes. I wanted something more stable, something that does not depend only on one threshold and can help me understand the model better. That is what led me toward predicted probabilities, because those values are generated directly by the model and do not change in the way predicted labels do. The predicted labels are only the final categories assigned after applying a threshold, but the probability itself is the deeper output of the model.

After that, I tried to understand these predicted probabilities in different ways. First through probability distribution, but that became difficult because there were too many buckets and too many values to compare properly. Then I moved to the ROC curve, which was much more meaningful in concept, because it checks model behavior across all thresholds and shows whether the model can separate positive outcomes from negative ones well or not. But even then, it remained difficult for me to use ROC directly for final judgment, because it still gave many values across many thresholds. I could see that the logistic model showed strong differences between TPR and FPR at many threshold points, but I still did not feel fully satisfied in the same way I was when I used accuracy earlier. So the main question became: is there any way to summarize all of this ROC information into one value that can help me compare the models more clearly?

That answer came through AUC, which is derived from the ROC curve itself. I was glad to find that such a metric exists, because ROC focuses on the model's behavior across all thresholds, and AUC turns that overall behavior into one single value. In simple terms, AUC tells how well the model ranks positive outcomes above negative outcomes. If I simplify it even more, it tells that if I randomly pick one positive case and one negative case, then AUC gives the probability that the model assigns a higher probability to the positive case than to the negative one. So if the AUC is 80%, that

means there is an 80% chance that the positive case will receive a higher predicted probability than the negative case. This makes AUC a useful overall measure of how confidently the model separates positives from negatives across all thresholds.

AUC works by looking at the ROC curve and measuring the area under it. Since the ROC curve is built from TPR and FPR across all thresholds, AUC summarizes how close that curve stays to the top-left region, where TPR is high and FPR is low. So the closer the curve moves toward that side, the larger the area under it becomes, and the better the model is at separating positive and negative outcomes.

I calculated AUC in two ways. First, I used the threshold table I had already created manually for the ROC analysis, where I used 21 thresholds from 0.0 to 1.0 in steps of 0.05. Second, I also used the direct BigQuery ML evaluation function, which calculates AUC using all thresholds internally. And this is where something important happened. My manual AUC still gave a picture similar to the accuracy analysis: logistic remained ahead, and the more complex models generally showed a drop from train to test. But the system AUC gave a broader picture. It showed that all models had strong test AUC values, and the difference between them became smaller than what I had earlier understood through only one threshold or even through the manual threshold set. So this gave me an important correction. Earlier, I was seeing the models mostly through threshold-based judgment. But once the full threshold behavior was considered, the gap between models looked much narrower.

This changed the question again. If all models are showing high confidence and strong separation ability across thresholds, then the next issue is no longer only confidence. The next issue is reliability. A model can still be highly confident and yet make dangerous mistakes. So the real question now became: even if the models are confident, which one behaves more reliably?

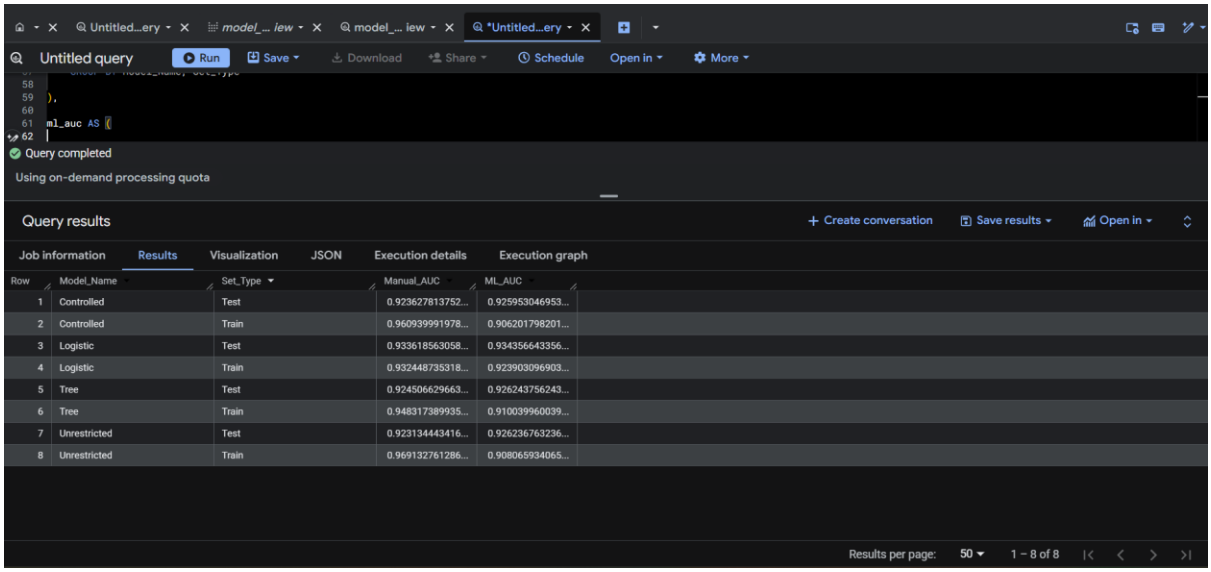
By reliability, I mean how the model behaves when it is confident. The most important part here is high-confidence wrong predictions. If a model is making mistakes with very high confidence, then that becomes a serious reliability risk, because such errors are more dangerous than low-confidence mistakes. So at this stage I moved away from only asking which model is more confident, and started asking which model is safer in its confidence.

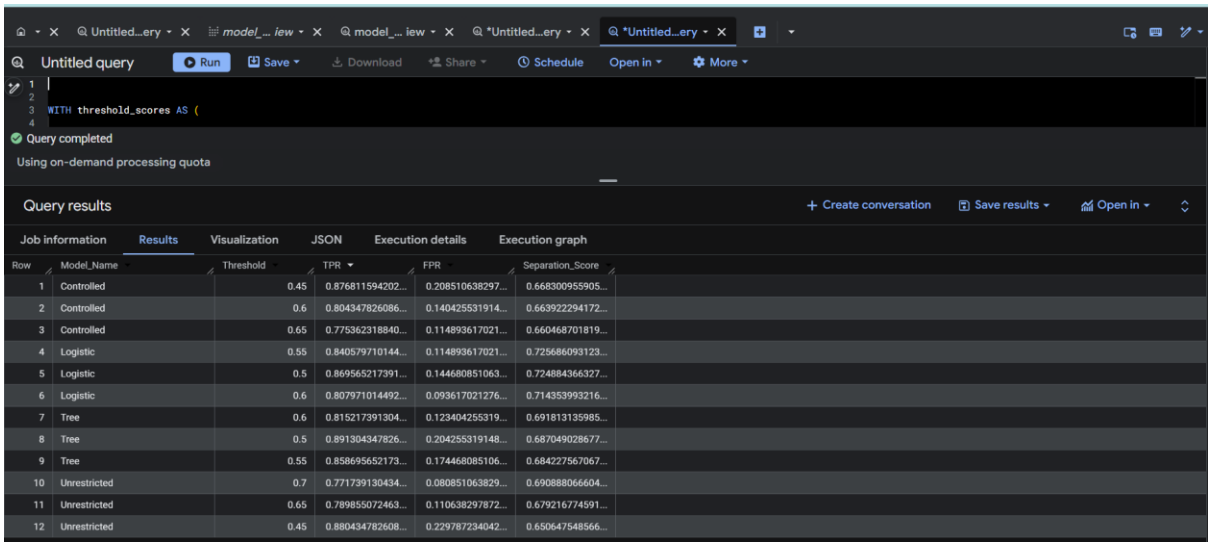
To do that, I used the TPR–FPR analysis I had already built and identified the common thresholds where the models were showing strong separation in the test data. The three most common threshold points were in the range of 0.50 to 0.60, so I selected 0.50, 0.55, and 0.60 for the reliability analysis. Then I created confidence bands. Predictions above 70% or below 30% were treated as high-confidence predictions, while probabilities between 30% and 70% were treated as low-confidence predictions.

The reliability analysis gave the clearest answer so far. Across all models, high-confidence correct predictions decreased from train to test, which is natural because test data is unseen. But the important difference was in how each model adjusted to this change. In the more complex models, the drop in high-confidence correct predictions was accompanied by an increase in wrong predictions, especially high-confidence wrong predictions. That suggests these models were learning some noise from the training data that did not carry well into the test data. But the logistic model behaved differently. Its high-confidence correct predictions also decreased, but it did not show the same rise in high-confidence wrong predictions. Instead, much of that shift moved toward low-confidence correct predictions. This means that on unseen data, the logistic model became more cautious rather than more dangerously confident.

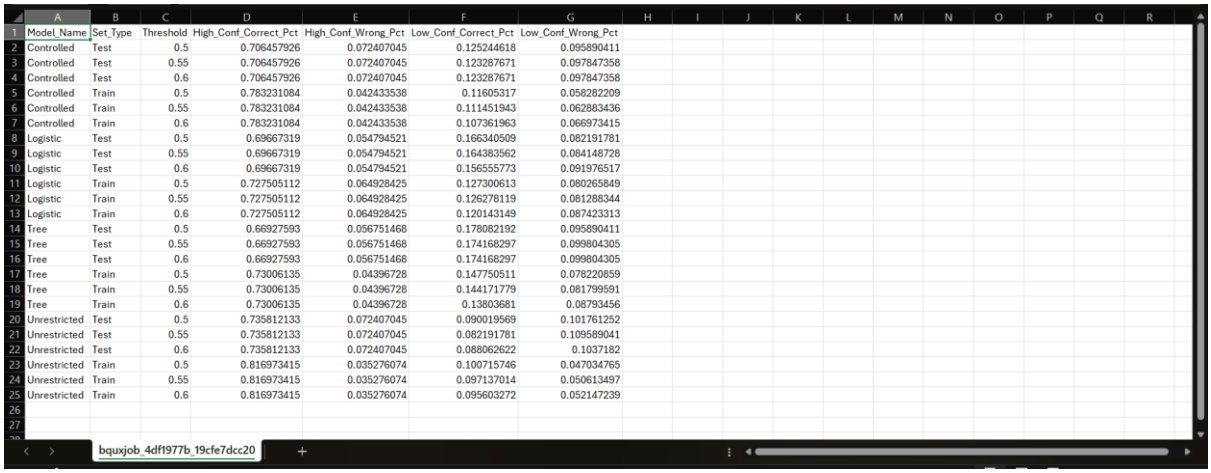
That is the key point. For final model choice, minimizing high-confidence wrong predictions matters more than maximizing high-confidence correct predictions. A model that stays a little more cautious on unseen data is safer and more dependable than a model that remains aggressive and makes more confident mistakes. From this point of view, the logistic model showed the most reliable behavior. It remained strong, but it also adjusted conservatively on test data, which makes it more trustworthy for practical use.

So overall, this question took the longest for me to answer, because confidence alone was not enough and even AUC did not create a decisive separation. AUC improved the understanding, but it did not settle the final choice by itself because the values remained close across the models. The final answer came from reliability. And from that angle, the logistic model is the most useful and most reliable model for this dataset. It gives strong performance, stays stable, and avoids the more dangerous pattern of high-confidence wrong predictions that appears more in the complex models.





Row	Model_Name	Threshold	TPR	FPR	Separation_Score
1	Controlled	0.45	0.876811594202...	0.208510638297...	0.668300955905...
2	Controlled	0.6	0.804347826086...	0.140425531914...	0.663922294172...
3	Controlled	0.65	0.775362318840...	0.114893617021...	0.660468701819...
4	Logistic	0.55	0.840579710144...	0.114893617021...	0.725686093123...
5	Logistic	0.5	0.869565217391...	0.144680851063...	0.724884366327...
6	Logistic	0.6	0.807971014492...	0.093617021276...	0.714353993216...
7	Tree	0.6	0.815217391304...	0.123404255319...	0.691813135985...
8	Tree	0.5	0.891304347826...	0.204255319148...	0.687049028677...
9	Tree	0.55	0.858695652173...	0.174468085106...	0.684227567067...
10	Unrestricted	0.7	0.771739130434...	0.080851063829...	0.690888066604...
11	Unrestricted	0.65	0.789855072463...	0.110638297872...	0.679216774591...
12	Unrestricted	0.45	0.880434782608...	0.229787234042...	0.650647548566...



Model_Name	Set_Type	Threshold	High_Conf_Correct_Pct	High_Conf_Wrong_Pct	Low_Conf_Correct_Pct	Low_Conf_Wrong_Pct
Controlled	Test	0.5	0.706457926	0.072407045	0.125244618	0.095890411
Controlled	Test	0.55	0.706457926	0.072407045	0.123287671	0.097847358
Controlled	Test	0.6	0.706457926	0.072407045	0.123287671	0.097847358
Controlled	Train	0.5	0.783231084	0.042433538	0.11605317	0.058282209
Controlled	Train	0.55	0.783231084	0.042433538	0.111451943	0.062883436
Controlled	Train	0.6	0.783231084	0.042433538	0.107361963	0.066973415
Logistic	Test	0.5	0.69667319	0.054794521	0.166340509	0.082191781
Logistic	Test	0.55	0.69667319	0.054794521	0.164383562	0.084148728
Logistic	Test	0.6	0.69667319	0.054794521	0.156555773	0.091976517
Logistic	Train	0.5	0.727505112	0.064928425	0.127300613	0.080265849
Logistic	Train	0.55	0.727505112	0.064928425	0.126278119	0.081288344
Logistic	Train	0.6	0.727505112	0.064928425	0.120143149	0.087423313
Tree	Test	0.5	0.66927593	0.056751468	0.178082192	0.095890411
Tree	Test	0.55	0.66927593	0.056751468	0.174168297	0.099804305
Tree	Test	0.6	0.66927593	0.056751468	0.174168297	0.099804305
Tree	Train	0.5	0.73006135	0.04396728	0.147750511	0.078220859
Tree	Train	0.55	0.73006135	0.04396728	0.144171779	0.081799591
Tree	Train	0.6	0.73006135	0.04396728	0.13803681	0.08793456
Unrestricted	Test	0.5	0.735812133	0.072407045	0.090019569	0.101761252
Unrestricted	Test	0.55	0.735812133	0.072407045	0.082191781	0.109589041
Unrestricted	Test	0.6	0.735812133	0.072407045	0.088062622	0.1037182
Unrestricted	Train	0.5	0.816973415	0.035276074	0.100715746	0.047034765
Unrestricted	Train	0.55	0.816973415	0.035276074	0.097137014	0.050613497
Unrestricted	Train	0.6	0.816973415	0.035276074	0.095603272	0.052147239

Answer to the fifth major question -

This was one of the toughest parts of the project for me, because accuracy alone was not enough, and the other metrics I moved into were much more complicated to understand. But that difficulty itself was important, because it showed that making the final model choice was not as simple as picking the highest accuracy value.

The first thing I understood was that accuracy could not be the final answer, because it depends on the threshold used to convert probabilities into categories. After that, I moved toward predicted probabilities, because those are the actual outputs generated by the model. But the probability distribution was too opaque to give a clear judgment, as there were too many buckets and the values were too close to each other. Then I moved into ROC, which was more meaningful because it checked the model across many thresholds, but it was still difficult to use directly for a simple final comparison. After that, AUC helped by summarizing the ROC into one value and gave a better overall view of the confidence of each model across thresholds.

But even then, the final answer did not fully come from confidence alone. What became clear was that most models were quite confident in separating positive and negative outcomes, and their AUC values were also strong. So the real difference had to be found somewhere deeper. That is where the reliability analysis became the most important part. Instead of asking only which model is more confident, I started asking which model behaves more safely and more dependably when it is confident.

That reliability analysis gave the clearest answer. The logistic model turned out to be the most reliable model, because while other models tended to capture more noise from the training dataset and reflect that less effectively in the test data, the logistic model behaved more conservatively. It reduced the risk of high-confidence wrong predictions and adjusted better on unseen data. So in the end, the final conclusion of this major question is that model evaluation should not be based only on accuracy. It should also include probability-based evaluation, ROC/AUC-based confidence understanding, and most importantly, reliability analysis. And from all of these together, the logistic model emerges as the most useful and reliable model for this dataset.

Answer to the Problem Statement -

This project was done to answer one main question: can machine learning be used on daily financial data in a meaningful way, and can it extract relationships or information that are useful further on? From the overall analysis, I can say that the answer is yes, but with an important condition.

The models used in this project did show that daily financial data contains meaningful structure. Across regression, classification, and tree-based models, the results remained stable between training and test data, which shows that the models were not working randomly. The relationship between the selected stocks and the Nifty target was strong enough to be captured by the models, and this itself shows that ML methods can analyze and interpret this kind of financial data in a useful way.

At the same time, the project also shows that usefulness does not increase just because the model becomes more complex. The simpler directional model, especially logistic regression, gave the most reliable result overall. More complex models did not collapse, but they also did not add enough extra usefulness, and in many cases they started capturing more noise from the training data. So this project also makes one thing clear: better complexity does not automatically mean better practical value.

Another important learning from the project is that the current limitation is not that ML cannot work on financial data. The limitation is more about the feature structure being used. With same-day return-based features, the models were able to capture useful signal and stable relationships, but to improve the quality of the result further, better

feature design will be needed. That can include lag-based variables, time-based patterns, regime-sensitive structure, or broader market and macro-related features.

So overall, this project shows that ML can be meaningfully applied to daily financial data, and it can extract useful relationships from it. But the real improvement going forward will not come only from using more advanced models. It will come from building better features on top of the financial data so that the models can understand deeper structure more effectively.